arXiv:2605.23096v2 [cs.CR] 25 Aug 2026

Encrypted Neural Networks without Overflows

Philipp Kern
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: philipp.kern@kit.edu

Lorenzo Rovida
Polytechnic University of Turin
Turin, Italy
Email: lorenzo.rovida@polito.it

Samuel Teuber
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: teuber@kit.edu

Edoardo Manino
The University of Manchester
Manchester, UK
Email: edoardo.manino@manchester.ac.uk

Carsten Sinz
Karlsruhe University of Applied Sciences
Karlsruhe, Germany
Email: carsten.sinz@h-ka.de

Alberto Leporati
University of Milan-Bicocca
Milan, Italy
Email: alberto.leporati@unimib.it

***Abstract*—The popular Cheon-Kim-Kim-Song (CKKS) scheme enables efficient private inference in neural networks by evaluating them on encrypted data. Since CKKS only supports addition, multiplication, and array rotation operations, turning neural networks into CKKS circuits requires approximating all activation functions (e.g. ReLU) with polynomials over fixed input ranges. Traditionally, these ranges are estimated via heuristic sampling techniques. In this paper, we empirically demonstrate that sampled ranges leave the CKKS network exposed to overflow events, i.e. legitimate inputs that exceed the ranges of the circuit and cause corrupted outputs, and propose an improved design technique that completely avoids them. First, we propose a practical algorithm to induce overflow events in a CKKS circuit, i.e. we automatically craft legitimate inputs that exceed the design ranges within the circuit and cause corrupted outputs. As an example, our algorithm can quickly corrupt around 47% of benign inputs in a CIFAR10 neural network. Second, we propose an end-to-end formal verification approach that constructs executable CKKS circuits. These circuits are guaranteed to be overflow-free and come with certified error bounds on the output difference w.r.t. the original neural network. Our experiments demonstrate that our approach supports a larger set of neural network architectures than prior certified techniques, while also delivering executable CKKS artifacts.**

1. Introduction

Private inference schemes based on fully-homomorphic encryption (FHE) are designed to delegate the bulk of the computation to a third-party, without revealing the data itself [1]. This is especially relevant for the so-called machine learning as a service (MLaaS) business model, where clients buy access to a machine learning model and run it on third-party servers [2]. An ideal FHE private inference scheme would encrypt the client data before transmission to the server, run the full inference over encrypted data, and only decrypt the output once it is transmitted back to the client. That way, the third-party server does never access the plaintext data, thus guaranteeing its confidentiality.

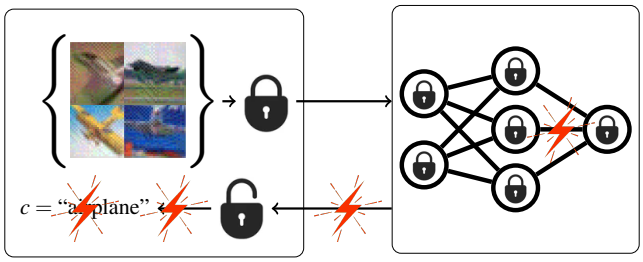


Figure 1: FHE allows neural network inference on encrypted data. However, some legitimate inputs can exceed the expected ranges of the FHE circuit, thus corrupting the output.

Several FHE schemes have been designed to realize this vision, including BGV [3], BFV [4], [5] and TFHE [6]. Among them, the CKKS scheme [7] has emerged as the de facto standard for efficiently implementing private neural inference, as it allows to natively encode vectors of complex numbers and perform massive arithmetic operations in parallel. Recent work demonstrates that executing deep convolutional networks [8] and large language models [9], [10] with CKKS is possible. However, the CKKS scheme imposes severe restrictions on the arithmetic operations available over the ciphertext: it only supports addition, multiplication and array rotation. Thus, existing CKKS inference frameworks must replace common non-linear activation functions like the ReLU with (high degree) polynomials within a certain interval range [11], [12].

However, there is a major weakness in modern CKKS designs for neural networks (NNs): there exists a sizeable set of legitimate inputs that make the CKKS network arbitrarily diverge from its reference plaintext implementation. This is because the polynomial approximations of non-linear

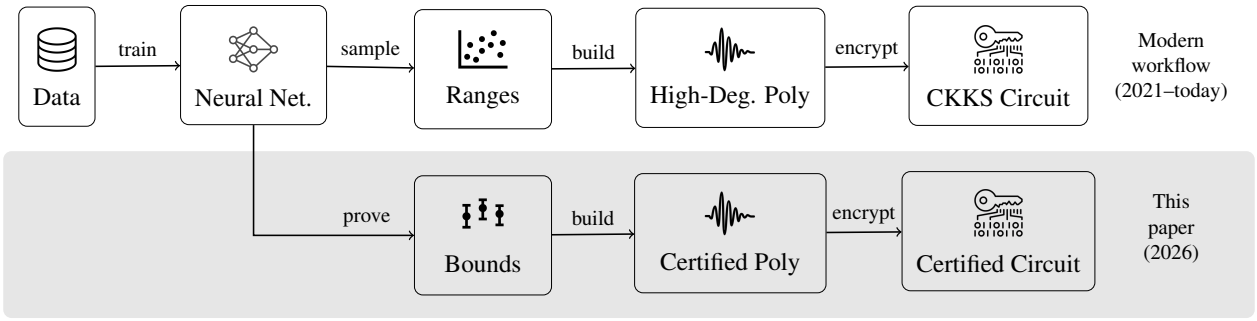


Figure 2: Designing CKKS neural networks requires approximating activations within their expected ranges. This paper avoids overflow events by computing provable bounds and certified polynomials.

activation functions are only tight for a limited range of values [13], [14]. A specific input needs only to exceed the range of a *single* activation to cause a cascade of overflows through the CKKS circuit and yield unrecoverable output values. We call this phenomenon an *overflow event* and note that it breaks the property of availability for the CKKS network: some inputs, no matter how reasonable-looking, will not produce a usable inference result (see Figure 1).

To address this challenge, we introduce a framework for the *certified* design of FHE NNs. Our framework guarantees the absence of overflow events, without any noticeable increase in the size of the FHE circuit. In detail, we propose the following contributions:

- We present an efficient algorithm to find inputs that exceed the expected ranges of NNs with polynomial activations. With it, we demonstrate that the modern design of CKKS NNs is vulnerable to overflows. Our empirical result shows that even small perturbations of benign inputs can cause up to 47% of unrecoverable outputs for a CIFAR10 network.
- To mitigate this, we introduce a differential verification framework that can programmatically compute certified bounds on the value range of all neurons in the NN and on the approximation error of the output. The framework can handle both analytic (e.g. GELU) and non-analytic activation functions (e.g. ReLU), fully-connected and convolutional layers.
- We give theoretical proofs that our bounds are sound for any possible input, i.e. both benign and adversarial, and thus can be used at design time to eliminate the possibility of overflows. We also show that NNs with analytical activation functions (e.g. GELUs) yield an asymptotically smaller approximation error than NNs with non-analytical ones (e.g. ReLUs). This gives theoretical grounding to a recent empirical observation reported by Ebel *et al.* [12].
- We further improve the design of CKKS NNs by allowing for the polynomial activations of different neurons in the same layer to be different, as opposed to uniform approximations in existing work. We thus generalize the Paterson-Stockmeyer algorithm [15], [16] from evaluating (ciphertext $\times$ constant) to (ciphertext $\times$ plaintext). Our experiments show that a heterogeneous design can achieve at least a $4.7\times$ reduction in error compared to the uniform one (as shown in Figure 8).

We provide an open-source implementation[1] of our framework, based on OpenFHE [17], which includes construction of certified polynomials and conversion to CKKS NNs.

Overview. Section 2 provides the necessary background on FHE (in particular CKKS) as well as NNs, their verification and their polynomial approximation. Section 3 introduces the notion of overflow events and presents an efficient algorithm for the discovery of overflowing inputs. Section 4 describes our methodology for the certified design of overflow-free CKKS circuits. Section 5 provides a theoretical argument for the usage of GELU activation functions and Section 6 describes how our approximations can be efficiently implemented in CKKS circuits. We evaluate our approach in Section 7.

2. Background

State-of-the-art FHE NNs, especially those based on the CKKS scheme, are typically constructed in a multi-stage process, as shown in the upper part of Figure 2: a regular neural network $f(\mathbf{x})$ is first approximated as a high-degree polynomial $f_\pi(\mathbf{x})$, after which the corresponding CKKS circuit $f_{\text{CKKS}}(\mathbf{x})$ can be built. Throughout the process, several sources of error are introduced:

$$\begin{aligned} f_{\text{CKKS}}(\mathbf{x}) &= f_\pi(\mathbf{x}) + \boldsymbol{e}_{\text{CKKS}}(\mathbf{x}) \\ &= f(\mathbf{x}) + \boldsymbol{e}_\pi(\mathbf{x}) + \boldsymbol{e}_{\text{CKKS}}(\mathbf{x}) \ . \end{aligned} \tag{1}$$

where $\boldsymbol{e}_\pi(\mathbf{x}) = f_\pi(\mathbf{x}) - f(\mathbf{x})$ is the *deterministic* polynomial approximation error stemming from the approximate representation of activation functions in f_π, while $\boldsymbol{e}_{\text{CKKS}}(\mathbf{x})$ is the stochastic error stemming from the CKKS encryption.

In the remainder of the section, we will characterise these sources of errors, from the encryption itself to polynomial approximation and range estimation.

1. Refer to https://github.com/lorenzorovida/encrypted-neural-networks-without-overflows

2.1. Fully Homomorphic Encryption

We begin by introducing some abstract definitions of Fully Homomorphic Encryption (FHE) schemes reported in the literature [18], [19] and extended to our needs.

Definition 2.1 (FHE Scheme)**.** A (public-key) homomorphic encryption scheme with plaintext space $\mathcal{M}$, ciphertext space $\mathcal{C}$, public key space $\mathcal{PK}$, secret-key space $\mathcal{SK}$, and space of evaluatable circuits $\mathcal{L}$ is a tuple of four probabilistic polynomial-time algorithms:

$$\mathsf{KeyGen} : 1^{\mathbb{N}} \to \mathcal{PK} \times \mathcal{SK} \quad \mathsf{Enc} : \mathcal{PK} \times \mathcal{M} \to \mathcal{C}$$
$$\mathsf{Eval} : \mathcal{PK} \times \mathcal{L} \times \mathcal{C} \qquad \mathsf{Dec} : \mathcal{SK} \times \mathcal{C} \to \mathcal{M} \to \mathcal{C}$$

In particular, in this work we will be dealing with *approximate-FHE*, i.e., such that $\mathsf{Dec}(\mathsf{Enc}(m)) \approx m$ for any $m \in \mathcal{M}$. Since the result of a decryption is approximate, standard correctness notions cannot be applied here. We thus first introduce a definition related to the impact of such approximation.

Definition 2.2 (Ciphertext Error [18])**.** Let $\prod = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$ be an FHE scheme with message space $\mathcal{M} \subseteq \tilde{\mathcal{M}}$, which is a normed space with norm $\|\cdot\| : \tilde{\mathcal{M}} \to \mathbb{R}_{\geq 0}$. For any ciphertext ct, secret key sk, and message m, the *ciphertext error* of $(\mathsf{ct}, m, \mathsf{sk})$ is defined to be

$$\mathsf{Error}(\mathsf{ct}, m, \mathsf{sk}) = \|\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) - m\|.$$

Note that we will conveniently require for $m_1, \ldots, m_k$ to be bounded, and this can be captured in the above definition by choosing $\mathcal{M}$ to be a set of bounded norm. It turns out that this definition is compatible with neural network verification (see Section 2.4.2) which also requires inputs $m \in \mathcal{M}$ with bounded norm.

For this work, it suffices to assume static correctness, i.e., we assume a statically computable function $\mathsf{Estimate}$ that bounds the ciphertext error after circuit execution:

Definition 2.3 (Static Approximate Correctness [18])**.** Let $\prod = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$ be an FHE scheme with message space $\mathcal{M} \subseteq \tilde{\mathcal{M}}$, which is a normed space with norm $\|\cdot\| : \tilde{\mathcal{M}} \to \mathbb{R}_{\geq 0}$. Let $\mathcal{L}$ be a space of circuits, $\mathcal{L}_k \subseteq \mathcal{L}$ the subset of parity k circuits, and let $\mathsf{Estimate} : \sqcup_{k \in \mathbb{N}} \mathcal{L}_k \times \mathbb{R}_{\geq 0}^k \to \mathbb{R}_{\geq 0}$ be an efficiently computable function. We call the tuple $\tilde{\prod} = (\prod, \mathsf{Estimate})$ a *statically approximate FHE scheme* if for all $k \in \mathbb{N}$, for all $\mathcal{C} \in \mathcal{L}_k$, for all $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\kappa)$, if $\mathsf{ct}_1, \ldots, \mathsf{ct}_k$ are the encryptions of $m_1, \ldots, m_k$ with $\mathrm{Error}(\mathsf{ct}_i, m_i, \mathsf{sk}) \leq t_i$, then

$$\mathsf{Error}(\mathsf{Eval}_{\mathsf{pk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_k), C(m_1, \ldots, m_k), \mathsf{sk}) \leq \mathsf{Estimate}(C, t_1, \ldots, t_k).$$

At the core of this definition is the $\mathsf{Estimate}$ function, which can be computed using different approaches [18], [19]. In practice, it acts as a tool to measure how far the CKKS circuit diverges from the plaintext one.

2.2. The CKKS Scheme

The Cheon–Kim–Kim–Song (CKKS) scheme [7] currently represents the standard framework to efficiently handle complex (and real) arithmetic in FHE. Similar to other FHE schemes, it is lattice-based and has a security reduction to the hardness of the Ring Learning with Errors (RLWE) problem [20]. This is a structured version of the original Learning with Errors (LWE) [21] introduced for efficiency reasons.

2.2.1. Basics of the Scheme. Let $N = 2^k$ for some integer $k > 0$ and let $Q > 0$ be some large modulus. Let $\mathsf{DFT} : \mathbb{R}[X]/(X^N + 1) \to \mathbb{C}^{N/2}$ be a discrete Fourier transform (DFT) defined as $\boldsymbol{m}(X) \mapsto (\boldsymbol{m}(\zeta^{5^i}))_{0 \leq i < N/2}$,[2] and let $\mathsf{iDFT} : \mathbb{C}^{N/2} \to \mathbb{R}[X]/(X^N + 1)$ be its inverse, with ζ being a $2N$-th primitive root of unity. The cleartext space of CKKS is $\mathbb{C}^{N/2}$, the plaintext space (i.e., after the encoding) is $\mathcal{R} := \mathbb{Z}[X]/(X^N + 1)$, and the ciphertext space is $\mathcal{R}_Q^2$, where $\mathcal{R}_Q$ is the quotient ring $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$.

Definition 2.4 (Ring Learning with Errors (RLWE) [20])**.** Let χ_s, χ_e be distributions over $\mathcal{R}_Q$.The goal of the RLWE problem is to distinguish $(\boldsymbol{a}, \boldsymbol{x})$ from $(\boldsymbol{a}, \boldsymbol{as} + \boldsymbol{e})$ for uniformly random $\boldsymbol{a}, \boldsymbol{x} \in \mathcal{R}_Q$, $\boldsymbol{e} \leftarrow \chi_{\boldsymbol{e}}$, and $\boldsymbol{s} \leftarrow \chi_{\boldsymbol{s}}$. The $\mathrm{RLWE}_{N,Q,\chi_e,\chi_s}$ assumption is that solving the $\mathrm{RLWE}_{N,Q,\chi_e,\chi_s}$ problem is computationally unfeasible.

In order to make the cleartext space compatible with RLWE encryption (which takes a message in $\mathcal{R}$ and returns a ciphertext in $\mathcal{R}_Q^2$, the CKKS scheme uses (the inverse of) a discrete Fourier transform encoding that moves a plaintext from $\mathbb{C}^{N/2}$ to $\mathcal{R}$ as follows:

$$\underbrace{\mathbb{C}^{N/2}}_{\text{Cleartext}} \xrightarrow[\text{Encoding}]{\text{Rounded iDFT}} \underbrace{\mathcal{R}}_{\text{Plaintext}} \xrightarrow[\text{Encryption}]{\text{RLWE}} \underbrace{\mathcal{R}_Q^2}_{\text{Ciphertext}},$$

where the rounding follows ideas from Lyubashevsky, Peikert and Regev [20, Section 2]. As it will be shown later, this rounding operation implies that we are required to scale the message by some large factor $\Delta > 0$ to preserve the most significant digits. In practice, numbers behave similarly to a fixed-point system. The scaling factor Δ is the key parameter governing computational precision: it yields greater accuracy but comes at the cost of increasing the ciphertext scale from Δ to Δ^2 after each multiplication.

To tackle this, CKKS introduces a moduli chain $Q_\ell = \prod_{i=0}^{\ell} q_i$, with $q_i \approx \Delta$, which enables a technique called *rescaling*: after a multiplication (whose result has scale Δ^2), the scale is reduced back to Δ by dividing by q_i, where i is the current multiplicative level. However, each rescaling operation consumes one modulus from the chain, so the depth of circuits that can be evaluated is bounded by the number of available q_i (number of levels ℓ) — unless additional techniques such as bootstrapping [22] are employed. The homomorphisms induced by (i)DFT

$$a + b = \mathsf{DFT}(\mathsf{iDFT}(a) + \mathrm{iDFT}(b)) \quad \text{and}$$
$$a \odot b = \mathsf{DFT}(\mathsf{iDFT}(a) \cdot \mathsf{iDFT}(b)),$$

2. $(1, 5, \ldots, 2^{N/2} - 3)$ is a cyclic subgroup of the multiplicative group $\mathbb{Z}_N^\times$ generated by 5. Rotations are thus enabled via $X \mapsto X^5$ [22].

where $\odot$ is the slot-wise multiplication, naturally induce a Single-instruction multiple data (SIMD) computational paradigm on the plaintext space by simply adding/multiplying ciphertexts in the iDFT domain. We can summarize the main operations of the scheme in what follows.

- $\mathsf{CKKS.Setup}(1^\lambda, N, \ell)$. Given a security parameter λ, a ring dimension N, and a multiplicative circuit depth ℓ, choose a modulus chain $Q_\ell = \prod_{i=0}^{\ell} q_i$, a secret key distribution χ_s, an error distribution χ_e, and a scaling factor $\Delta \in \mathbb{Z}_{>0}$. Output the public parameter tuple
$$\boldsymbol{t} := \big(N,\ \Delta,\ Q_\ell,\ \chi_s,\ \chi_e\big).$$
- $\mathsf{CKKS.KeyGen}(\boldsymbol{t})$. Given the public parameter tuple $\mathbf{t}$, sample $\boldsymbol{s} \sim \chi_s$ and $\mathbf{a} \leftarrow \mathcal{R}_{Q_\ell}$ uniformly, and $\mathbf{e} \sim \chi_e$. Output the secret and public keys
$$\mathsf{sk} := \boldsymbol{s} \in \mathcal{R}, \quad \mathsf{pk} := \big(-\boldsymbol{a}\cdot\boldsymbol{s} + \boldsymbol{e},\ \boldsymbol{a}\big) \in \mathcal{R}^2_{Q_\ell}.$$
Note that $\mathsf{pk}_0 + \mathsf{pk}_1 \cdot \boldsymbol{s} = \boldsymbol{e} \approx 0$, so the public key is a noisy encryption of zero under $\boldsymbol{s}$.
- $\mathsf{CKKS.Encode}(\mathbf{m})$. Return: $\lfloor \Delta \cdot \mathsf{iDFT}(\boldsymbol{m}) \rceil \in \mathcal{R}$.
- $\mathsf{CKKS.Decode}(\boldsymbol{p})$. Return: $1/\Delta \cdot \mathsf{DFT}(\boldsymbol{p}) \in \mathbb{C}^{N/2}$.
- $\mathsf{CKKS.Encrypt}(\boldsymbol{p})$. Given a plaintext $\boldsymbol{p} \in \mathcal{R}_{Q_\ell}$ and the public key pk, sample a random mask $\boldsymbol{u} \sim \chi_s$ and noise terms $\boldsymbol{e}_0, \boldsymbol{e}_1 \sim \chi_e$. Output the ciphertext
$$\big([pk_0\,\boldsymbol{u} + \boldsymbol{e}_0 + \mathbf{p}]_{Q_\ell},\ [pk_1\,\boldsymbol{u} + \boldsymbol{e}_1]_{Q_\ell}\big) \in \mathcal{R}^2_{Q_\ell}.$$
- $\mathsf{CKKS.Decrypt}(\boldsymbol{c})$. Given a ciphertext $\boldsymbol{c} = (c_0, c_1) \in \mathcal{R}^2_{Q_\ell}$ and the secret key $\boldsymbol{s}$, compute the inner product $c_0 + c_1\,\mathbf{s}$. Output the approximate plaintext
$$\begin{aligned} c_0 + c_1\,\mathbf{s} &= \big(pk_0\,\boldsymbol{u} + \boldsymbol{e}_0 + \boldsymbol{p}\big) + \big(pk_1\,\boldsymbol{u} + \boldsymbol{e}_1\big)\boldsymbol{s} \\ &= \underbrace{\boldsymbol{e}\cdot\boldsymbol{u} + \boldsymbol{e}_0 + \boldsymbol{e}_1\,\boldsymbol{s}}_{\text{small noise}} + \boldsymbol{p} \approx \boldsymbol{p} \in \mathcal{R}_{Q_\ell}. \end{aligned}$$
Notice that the term indicated above with *small noise* is precisely the $\boldsymbol{e}_{\mathrm{CKKS}}(\boldsymbol{p})$ term abstractly introduced in Eq.(1).

Encryption and decryption follow from standard RLWE [20]. Notice that the CKKS noise is mainly given by two factors: the rounding performed in the encoding and the error injected during encryption.

2.2.2. Homomorphic Operations. The CKKS scheme only supports a limited number of homomorphic operations [7], namely:

- $\mathsf{CKKS.Add}(\boldsymbol{c}, \boldsymbol{c}')$. Given two ciphertexts $\boldsymbol{c}, \boldsymbol{c}' \in \mathcal{R}^2_{Q_\ell}$, output the component-wise sum
$$\boldsymbol{c}_{\text{add}} := \boldsymbol{c} + \boldsymbol{c}' = (c_0 + c'_0,\ c_1 + c'_1) \in \mathcal{R}^2_{Q_\ell},$$
satisfying $\mathsf{Decrypt}(\boldsymbol{c}_{\text{add}}) \approx \mathsf{Decrypt}(\boldsymbol{c}) + \mathsf{Decrypt}(\boldsymbol{c}')$.
- $\mathsf{CKKS.Mult}(\boldsymbol{c}, \boldsymbol{c}')$. Given two ciphertexts $\boldsymbol{c}, \boldsymbol{c}' \in \mathcal{R}^2_{Q_\ell}$, compute the degree-2 tensor product
$$(c_0 c'_0,\ c_0 c'_1 + c_1 c'_0,\ c_1 c'_1) \in \mathcal{R}^3_{Q_\ell},$$
and apply $\mathsf{KeySwitch}(\cdot, \mathsf{sk}^2 \to \mathsf{sk})$ to reduce back to two components. Output
$$\boldsymbol{c}_{\text{mul}} \in \mathcal{R}^2_{Q_\ell},$$
satisfying $\mathsf{Decrypt}(\boldsymbol{c}_{\text{mul}}) \approx \mathsf{Decrypt}(\boldsymbol{c}) \cdot \mathsf{Decrypt}(\boldsymbol{c}')$. Note that the resulting scaling factor is Δ^2; a subsequent call to Rescale is typically needed to restore it to Δ.
- $\mathsf{CKKS.KeySwitch}(\boldsymbol{c}, \mathsf{evk})$. Given a ciphertext $\boldsymbol{c} = (c_0, c_1, c_2) \in \mathcal{R}^3_{Q_\ell}$ encrypted under sk^2 and an evaluation key evk — a public encryption of sk^2 under sk — decompose c_2 into a base-P representation (as in [23]) $c_2 = \sum_j c_2^{(j)} P^j$ and absorb it into the first two components using evk. Output the key-switched ciphertext
$$\big(c_0 + \textstyle\sum_j c_2^{(j)}\,\mathsf{evk}_0^{(j)},\ c_1 + \sum_j c_2^{(j)}\,\mathsf{evk}_1^{(j)}\big) \in \mathcal{R}^2_{Q_\ell},$$
which encrypts the same plaintext under sk with a controlled increase in noise.
- $\mathsf{CKKS.Rescale}(\boldsymbol{c})$. Given a ciphertext $\boldsymbol{c} \in \mathcal{R}^2_{Q_\ell}$ with scaling factor Δ^2, drop the last modulus q_ℓ from the chain and divide. Output
$$\lfloor \Delta^{-1} \cdot \boldsymbol{c} \rceil \in \mathcal{R}^2_{Q_{\ell-1}},$$
which restores the scaling factor to Δ at the cost of consuming one level of the modulus chain.
- $\mathsf{CKKS.Rotate}(\boldsymbol{c}, i)$. Given a ciphertext $\boldsymbol{c} \in \mathcal{R}^2_{Q_\ell}$ and a rotation index $i \in \mathbb{Z}$, apply the Galois automorphism $\sigma_i \colon X \mapsto X^{5^i \bmod 2N}$ component-wise to obtain $\sigma_i(\boldsymbol{c}) \in \mathcal{R}^2_{Q_\ell}$, and apply $\mathsf{KeySwitch}(\cdot, \sigma_i(\mathsf{sk}) \to \mathsf{sk})$ using the corresponding rotation key. Output
$$\boldsymbol{c}_{\text{rot}} \in \mathcal{R}^2_{Q_\ell},$$
which satisfies $\mathsf{Decrypt}(\boldsymbol{c}_{\text{rot}}) \approx \rho^i(\mathsf{Decrypt}(\boldsymbol{c}))$, where ρ^i denotes a cyclic shift of the plaintext slot vector by i positions.

Operation	Noise Growth	Latency	Levels Consumed
$\mathsf{Add}(\boldsymbol{c}_1, \boldsymbol{c}_2)$	•	•	unchanged
$\mathsf{Sub}(\boldsymbol{c}_1, \boldsymbol{c}_2)$	•	•	unchanged
$\mathsf{Mul}(\boldsymbol{c}_1, \boldsymbol{c}_2)$	•••	•••	1
$\mathsf{Mul}(\boldsymbol{c}, \boldsymbol{p})$	••	•	1
$\mathsf{Rotate}(\boldsymbol{c}, i)$	•	•••	unchanged

TABLE 1: Overview on arithmetic and rotation operations in CKKS. • low, •• medium, ••• high.

We summarize the noise growth and the runtime of the available operations in Table 1. Nevertheless, we refer the interested reader to the RNS-CKKS implementation by Kim, Papadimitriou and Polyakov [24] used in this work.

2.3. Polynomial Networks

Due to the restricted set of operations available in the CKKS scheme (see Section 2.2.2), a neural network f must be converted into a sequence of addition, multiplication, rotations, and rescaling operations. More precisely, consider a standard feed-forward neural network:

Definition 2.5 (Feed-Forward Neural Network)**.** A *neural network* computes a function $f : \mathcal{I} \to \mathbb{R}^O$ that maps

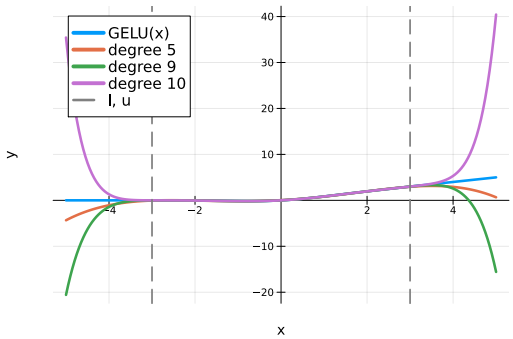


Figure 3: Polynomial approximations diverge to $\pm\infty$ outside of the approximation interval.

vectors from the I-dimensional input space $\mathcal{I} \subseteq \mathbb{R}^I$ to an O-dimensional output space. A feed-forward architecture is a composition of layers $l \in [1, L]$ defined as $\mathbf{z}^{(l)} = \sigma^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right) = \sigma^{(l)}\left(W^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}\right)$, where $\sigma^{(l)}$ is an activation function, $\mathbf{x} = \mathbf{z}^{(0)} \in \mathcal{I}$ is the network input, and $\mathbf{z}^{(L)} \in \mathbb{R}^O$ the network output. We denote by $f^{(l)}(\mathbf{x})$ the value of $\tilde{\mathbf{z}}^{(l)}$ at layer l for input $\mathbf{x}$.

While the linear term $W^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$ can be natively implemented with the operations listed above, the activations $\sigma^{(l)}$ must be replaced with polynomial functions as follows:

Definition 2.6 (Polynomial Neural Network)**.** A *polynomial neural network* computes a function $f_\pi : \mathcal{I} \to \mathbb{R}^O$ obtained from a neural network $f : \mathcal{I} \to \mathbb{R}^O$ by replacing the activation functions $\sigma^{(l)}$ in Definition 2.5 by their polynomial approximation counterparts $\pi^{(l)}$ (see Sec. 4.1 for their construction). To differentiate the two networks, we rename the pre- and post-activation vectors from $\mathbf{z}$ to $\mathbf{y}$, i.e. $\mathbf{y}^{(l)} = \pi^{(l)}\left(\tilde{\mathbf{y}}^{(l)}\right) = \pi^{(l)}\left(W^{(l)}\mathbf{y}^{(l-1)} + \mathbf{b}^{(l)}\right)$ with $\mathbf{y}^{(0)} = \mathbf{z}^{(0)}$ and $\mathbf{y}^{(l)} \approx \mathbf{z}^{(l)}$ for all layers $l > 0$. We call $f_\pi^{(l)}(\mathbf{x})$ the value of $\tilde{\mathbf{y}}^{(l)}$ for input $\mathbf{x}$.

As per Equation (1), we call $e_\pi(\mathbf{x}) = f_\pi(\mathbf{x}) - f(\mathbf{x})$ the approximation error introduced by polynomial networks. The common strategy to reduce this source of error is the use of higher-degree polynomials, at the cost of higher multiplicative depth in the CKKS circuit and thus an increase in latency [11].

2.4. Range Estimation

To construct accurate polynomial approximations of the activation functions $\sigma^{(l)}$, we must bound the range of their input, i.e. their pre-activation values $f_\pi^{(l)}(\mathbf{x})$. Therefore, we need to identify intervals $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ for each layer l such that $f_\pi^{(l)}(\mathbf{x}) \in [\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ is satisfied for all inputs $\mathbf{x} \in \mathcal{I}$. If there exist inputs $\mathbf{x}$ where $f_\pi^{(l)}(\mathbf{x})$ is not in the computed approximation interval, the approximation may result in large errors and uncontrollable growth, eventually leading to arithmetic failures. This phenomenon is visible in the GELU approximations in Fig. 3 which are precise for the range $[-3, 3]$, but rapidly diverge outside of it.

2.4.1. Sampling-Based Design. The most common approach to compute ranges is based on observations from a dataset $\mathcal{D}$ sampled from the training data distribution. For intermediate layers l, bounds $\mathbf{l}^{(l)}, \mathbf{u}^{(l)}$ are computed such that $\mathbf{l}^{(l)} \leq f_\pi^{(l)}(\mathbf{x}) \leq \mathbf{u}^{(l)}$ holds for all $\mathbf{x} \in \mathcal{D}$.[3]

Such sampling-based estimates provide no guarantees outside the dataset $\mathcal{D}$ and may significantly underestimate the true ranges. Existing countermeasures include extending sampled ranges by a factor of two [12] or choosing a constant range of $[-c, c]$ for a suitable large constant $c > 0$ [25]. Others propose to reduce the impact of outliers on ranges by pre-multiplying the weights by orthogonal matrices [10].

In Section 3, we show that such countermeasures do not protect against all range violations, thus leading to catastrophic failures in the CKKS network.

2.4.2. Neural Network Verification. A principled alternative to the empirical estimation of ranges is the use of neural network verification techniques. For classical (non-polynomial) neural networks, there are numerous tools [26], [27], [28] that can solve constraint and optimization problems on the computational graph of a neural network. For example, such tools can be used on non-polynomial neural networks to compute pre-activation ranges by solving the following optimization problem for all $l \in [1, L]$:

$$\min / \max \quad \tilde{\mathbf{z}}^{(l)} \tag{2}$$
$$\text{s.t. } \mathbf{z}^{(0)} \in \mathcal{I} \text{ and } \begin{cases} \tilde{\mathbf{z}}^{(j)} = W^{(j)}\mathbf{z}^{(j-1)} + \mathbf{b}^{(j)} \\ \mathbf{z}^{(j)} = \sigma^{(j)}\left(\tilde{\mathbf{z}}^{(j)}\right) \end{cases} \text{for } j \leq l \,.$$

The progress in NN verification scalability is recorded in the yearly competition [29]. While this research field has seen significant advances in general, many state-of-the-art techniques are not readily applicable to (high-degree) polynomial NNs.

2.4.3. Differential Verification. Standard NN verifiers are also notoriously bad at the verification of bounds on the *difference* between two NN executions [30], [31], [32], i.e. $f_1(\mathbf{x}) - f_2(\mathbf{x})$ for distinct NNs f_1 and f_2. Hence, specialized solvers [30], [31], [32] propose *differential verification* propagating bounds on the execution difference *up to layer* l (i.e. $f_1^{(l)}(\mathbf{x}) - f_2^{(l)}(\mathbf{x})$) through the NN. For example, the difference $\tilde{\mathbf{\Delta}}^{(l)} = \tilde{\mathbf{y}}_1^{(l)} - \tilde{\mathbf{y}}_2^{(l)}$ between two pre-activation values can be expressed in terms of the difference $\mathbf{\Delta}^{(l-1)} = \mathbf{y}_1^{(l-1)} - \mathbf{y}_2^{(l-1)}$ between the results of the previous layer via the identity

$$\tilde{\mathbf{\Delta}}^{(l)} = \left(W_1^{(l)} - W_2^{(l)}\right)\mathbf{y}_2^{(l-1)} + W_1^{(l)}\mathbf{\Delta}^{(l-1)} + \left(\mathbf{b}_1^{(l)} - \mathbf{b}_2^{(l)}\right).$$

The post-activation difference can be similarly rewritten as

$$\mathbf{\Delta}^{(l)} = \sigma_1^{(l)}(\tilde{\mathbf{y}}_1^{(l)}) - \sigma_2^{(l)}(\tilde{\mathbf{y}}_2^{(l)}) = \sigma_1^{(l)}(\tilde{\mathbf{y}}_1^{(l)}) - \sigma_2^{(l)}(\tilde{\mathbf{y}}_1^{(l)} - \tilde{\mathbf{\Delta}}^{(l)}) \,.$$

3. For ease of implementation, the activation ranges are often estimated on the regular network $f^{(l)}(\mathbf{x})$ rather than $f_\pi^{(l)}(\mathbf{x})$.

While this allows to efficiently bound the difference between executions of classical NNs, this approach [31], [32] is also not directly applicable to high-degree polynomial NNs.

To address both of these limitations, a recent line of work leverages NN verification for sound range estimation and to bound the numerical error between a regular NN $f(\mathbf{x})$ and its polynomial variant $f_\pi(\mathbf{x})$ [13], [14]. LiGAR [13] allocates the optimal polynomial approximations for an NN under a given maximal error budget by extending the optimization problem in Equation (2) with bounded error variables by redefining $\mathbf{z}^{(j)}$ as follows:

$$\mathbf{z}^{(j)} = \sigma^{(j)}\left(\tilde{\mathbf{z}}^{(j)}\right) + \boldsymbol{\delta}^{(j)}\,, \tag{3}$$

where the admissible range of $\boldsymbol{\delta}^{(j)}$ is determined via an iterative refinement-cycle. Importantly, this approach only works with the degree of the best polynomial approximation, but does not actually compute the polynomial coefficients. Furthermore, it estimates the impact via (possibly conservative) Lipschitz constants.

In contrast, ZONOPOLY [14] does compute concrete polynomial coefficients and can prove tighter bounds than LIGAR. However, its verification strategy is based on zonotope propagation, which cannot compete with state-of-the-art NN verifiers with respect to computing tight pre-activation ranges. Additionally, both LIGAR and ZONOPOLY are only applicable to neural networks $f(\mathbf{x})$ with ReLU activations, and do not demonstrate whether their bounds translate to a robust CKKS circuit. These are severe limitations, as smoother activations can drastically reduce the approximation error (see Section 5) and designing a CKKS circuit that can accommodate robust polynomial approximations requires non-trivial work (see Section 6).

3. Overflow Events in CKKS Networks

At an abstract level, private inference over encrypted data consists of computing the *total* function $f_{\text{CKKS}}(\mathbf{x}) = f_\pi(\mathbf{x}) + \boldsymbol{e}_{\text{CKKS}}(\mathbf{x})$, which will always return *some* value. Any issue that may occur during the computation is by definition undetectable until the output is decrypted.

One such issue is that of a *plaintext overflow* [33]. More specifically, consider that the plaintext space of CKKS, i.e. $\mathcal{R}$ is embedded in $\mathcal{R}^2_{Q_\ell}$ for security reasons. This implies that, any coefficient in the plaintext polynomial being larger than $Q_\ell/2$ can corrupt the encoded message. In order to avoid an explosion in the small ciphertext error $\boldsymbol{e}_{\text{CKKS}}(\mathbf{x})$, the infinity norm of any plaintext message must remain smaller than half the modulus Q_ℓ in which it is encrypted. Whenever such condition is violated, the plaintext coefficients will wrap around Q_ℓ, thus corrupting the result:

Definition 3.1 (Plaintext Overflow)**.** For any ciphertext $\boldsymbol{c}$ encrypting $\boldsymbol{m} \in \mathcal{R}$, we say that $\boldsymbol{c}$ contains a *plaintext overflow* if the underlying $\boldsymbol{m}$ is such that $\|\boldsymbol{m}\|_\infty \geq Q_\ell/2$, where Q_ℓ is the RLWE modulus.

Plaintext overflows happen when the message to be represented grows too large. Here, we argue that evaluating high-degree polynomial activations may easily trigger such an event. Indeed, a single activation exceeding its approximation interval may suffice to yield values that quickly diverge to $\pm\infty$ (see Figure 3), thus rapidly exhausting the available modulus Q_ℓ. We formalise an overflow event as follows:

Definition 3.2 (Polynomial Overflow Events)**.** Let $f_\pi : \mathcal{I} \to \mathbb{R}^O$ be a polynomial network with expected ranges $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ for all layers l. An input $\mathbf{x} \in \mathcal{I}$ will cause an *overflow event on* f_π iff there is a layer l where $f_\pi^{(l)}(\mathbf{x}) \notin [\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$.

Note that there is no one-to-one correspondence between plaintext and polynomial overflows: the modulus Q_ℓ might be large enough to accommodate intermediate layers $f_\pi^{(l)}(\mathbf{x})$ that exceed their expected ranges $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ by a very small amount. However, more significant violations can exhaust the modulus and cause unrecoverable outputs. Indeed, we show how such inputs can easily be constructed in Section 3.2 and demonstrate how this issue can occur in practice in Section 3.3.

In contrast, we would like to design polynomial networks for which polynomial overflow events can never occur. That way, the plaintext message will always remain within its design constraints from Definition 3.1:

Definition 3.3 (Overflow Robustness)**.** A network f_π is *overflow robust* on $\mathcal{I}$ if no $\mathbf{x} \in \mathcal{I}$ can cause an overflow event on f_π.

We give one such construction in Section 4.

3.1. A Note on the Threat Model

Before introducing our method to find overflow events, let us clarify our assumptions on the threat model. Here, we are concerned with the *possibility* of overflow events. That is, an honest user providing a legitimate input which *accidentally* causes an overflow event. This can already happen when processing images from a different test set: for instance, training a handwritten digits classifier on the MNIST dataset and evaluating it on USPS (see Table 2).

Below, we additionally provide an auditing technique that searches for benign-looking inputs that cause overflows. While our technique constructs inputs in an adversarial fashion, these inputs could also be supplied in practice by an honest user.

3.2. Finding Overflows

To uncover overflow events in polynomial networks, we need to find legitimate inputs $\mathbf{x} \in \mathcal{I}$ that exceed the expected ranges $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$. To do so, we leverage classic techniques from adversarial example generation: for any given benign input $\mathbf{x} \in \mathcal{I}$, we construct a perturbed input $\mathbf{x}' = \gamma(\mathbf{x}, \boldsymbol{\delta})$. Here, the parameter $\boldsymbol{\delta}$ regulates the intensity and direction of local perturbations around $\mathbf{x}$. In our experiments (see Section 3.3), γ is a combination of small image rotations, small translations, and additive noise (see, e.g., Figure 4).

We then use projected gradient descent [34] to optimize $\boldsymbol{\delta}$ towards inputs $\mathbf{x}'$ that violate the expected ranges of f_π.

Since direct optimization on f_π is numerically unstable, we optimize with respect to f instead. Note that the internal activation values $f^{(l)}(\mathbf{x})$ and $f_\pi^{(l)}(\mathbf{x})$ are tightly related (until the latter violates its bound) and thus can act as a natural surrogate of each other. More formally, we maximize the distance between the internal activation values and the centre point of their expected ranges $(\mathbf{u}^{(l)} + \mathbf{l}^{(l)})/2$:

$$\sum_j \left\| \left(f^{(l)}(\gamma(\mathbf{x}, \boldsymbol{\delta})) \right)_j - \left(\mathbf{u}_j^{(l)} + \mathbf{l}_j^{(l)} \right) /2 \right\|_\infty . \quad (4)$$

As will be shown below, optimising over small perturbations $\boldsymbol{\delta}$ of benign inputs $\mathbf{x}$ generates realistic inputs causing overflows.

3.3. Overflow Events in Sampling-Based Networks

To demonstrate the vulnerability of sampling-based CKKS network construction to overflow events, we deploy the algorithm described above on classification NNs for tabular data (HELOC [35]), handwritten digit classification (MNIST [25]), and image classification (CIFAR [36]). We compare the vulnerability of a CKKS network constructed via sampled bounds to the vulnerability of an NN constructed with our certified approach described in Section 4. In both cases, we use GeLU activation functions and heterogeneous polynomials. Table 2 provides an overview and demonstrates that our approach presented in Section 4 yields overflow robust networks. Notably, the overflow-event rate yields an *upper bound* on the possible accuracy of the CKKS circuit: If p % of samples have an overflow event, then the accuracy of the circuit can be at most $100 - p$ % (as the remaining predictions cannot be decoded).

For HELOC we generate sampled bounds using the first half of the dataset and use the other half for finding overflows. We allow perturbations of 15% w.r.t. each feature's range. To discretize values, our adversarial procedure uses a straight-through-estimator [37]. We find three overflows on the sampled NN.

For MNIST we generate sampled bounds using the original MNIST dataset, then we search for overflow-inducing inputs on the USPS dataset [38] (same input configuration and task setting). Note that the USPS dataset already causes overflows in 5.18% of cases without the need of adding any input perturbations. When we allow rotations of $23°$, translation by 20% and L_∞-noise of up to $\varepsilon = 5/255$, we manage to increase the overflow rate to approx. 12.3% of cases, while still producing realistic-looking inputs.

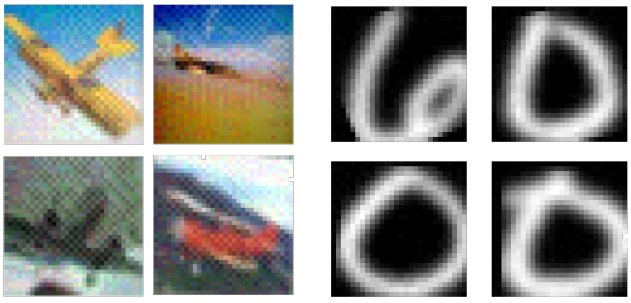


Figure 4: Realistic perturbations of CIFAR and MNIST images causing overflow events

For CIFAR, we generate sampled bounds using the original CIFAR dataset. We search for overflows on the independent CIFAR10.1 dataset [39], [40] (same input configuration and task setting). We allow rotations of $17°$ and L_∞ noise of up to $\varepsilon = 15/255$ (no translation due to non-black background in images). We find overflows in both a small and a deeper CIFAR NN. For the latter, we manage to cause an overflow on 46.7% of images (see Figure 4). This implies the CKKS circuit accuracy is below 53.3% on the evaluated dataset.

Reproduction in CKKS Circuit and Discussion. Our adversarial procedure initially only searches for perturbations that push the polynomial NN f_π outside its bounds. As discussed above, this overflow is likely to induce decoding errors for the CKKS circuit. We validated this assumption on a subset of MNIST inputs and showed that all overflow events on f_π lead to arithmetic errors in the CKKS circuit, which underscores the importance of eliminating overflows. Table 2 shows that our certified design introduced in the next section eliminates overflows without major impacts on accuracy. The subsequent sections will describe a principled approach for the construction of such certified CKKS circuits.

4. Avoiding Overflows by Certified Design

Given a neural network f, we propose a method to construct robust CKKS circuits in the sense of Definition 3.3, i.e. NNs f_π without overflows, while simultaneously bounding the output difference $\|f_\pi(\mathbf{x}) - f(\mathbf{x})\|_\infty$. Our approach transforms f into its polynomial version f_π by iteratively rewriting its layers. For each layer l, we first compute provable bounds such that $f_\pi^{(l)}(\mathbf{x}) \in [\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ for all $\mathbf{x} \in \mathcal{I}$ (Section 4.1). Linear layers then remain unchanged while activation functions are substituted by a polynomial approximation, obtained on the fly for the interval $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ using the Remez algorithm [41]. The algorithm relies on a bound for the maximum (local) error between the activation function and its approximation (Section 4.2) which is taken into account for the range analysis in subsequent layers. We additionally leverage differential verification via zonotopes [32] to compute bounds on the difference $\|f_\pi^{(l)}(\mathbf{x}) - f^{(l)}(\mathbf{x})\|_\infty$ (Section 4.3). Furthermore, we provide a direct translation from the generated polynomial network f_π to a CKKS circuit (Section 7.1).

4.1. Sound Range Analysis for Polynomial Networks

We construct the polynomial NN f_π from a given NN f by iteratively substituting a non-linear activation function $\sigma^{(l)}$ in layer l by its polynomial approximation $\pi^{(l)}$. The latter is computed by the Remez algorithm [41] which requires a method for computing a sound bound on $|\pi^{(l)}(\mathbf{x}) - \sigma^{(l)}(\mathbf{x})|$ (see Section 4.2) and an approximation range $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$.

TABLE 2: Comparison between sampled ranges and certified construction of CKKS networks. While the design-time accuracy is similar, the former are susceptible to overflows. These already happen naturally on MNIST, as soon as we change the test set, but our auditing technique (Found Overflows) reveals a larger percentage of them on both HELOC and CIFAR10.

		Design-Time Accuracy (higher is better)			**Naturally-Occurring Overflows** (lower is better)			**Found Overflows Events** (lower is better)		
Network	**Size**	**Dataset**	**Sampled**	**Certified**	**Dataset**	**Sampled**	**Certified**	**Dataset**	**Sampled**	**Certified**
HELOC	$[64, 32]$	HELOC (1)	**71.86%**	**71.86%**	HELOC (2)	**0.0%**	**0.0%**	HELOC (2)	0.05%	**0.0%**
MNIST	[256,256,256,256]	MNIST	**97.80%**	**97.80%**	USPS	5.18%	**0.0%**	USPS	12.30%	**0.0%**
CIFAR10	$[900, 392, 144, 256]$	CIFAR 10	**69.54%**	69.50%	CIFAR 10.1	**0.0%**	**0.0%**	CIFAR 10.1	0.10%	**0.0%**
	$[1.8\text{k}, 784, 288, 256]$	CIFAR 10	**77.28%**	75.13%	CIFAR 10.1	**0.0%**	**0.0%**	CIFAR 10.1	46.70%	**0.0%**

To compute valid ranges for $f_\pi^{(l)}$ (beyond the first polynomial layer) it is insufficient to analyse NN f in isolation, as its output bounds also depend on the polynomial approximation errors introduced in earlier layers. To this end, we follow the approach by Manino *et al.* [13] and compute sound ranges by optimizing the problem given in Equation (2) with the modification given in Equation (3). However, in contrast to prior work, we bound $\boldsymbol{\delta}^{(j)}$ a priori with the interval $[-\boldsymbol{\epsilon}^{(l)}, \boldsymbol{\epsilon}^{(l)}]$ where $\boldsymbol{\epsilon}^{(l)}$ is computed as seen in Section 4.2 (i.e., $|\pi^{(l)}(\mathbf{x}) - \sigma^{(l)}(\mathbf{x})| \leq \boldsymbol{\epsilon}^{(l)}$).

As the resulting optimization problem only contains regular activations $\sigma^{(l)}$ (instead of polynomials $\pi^{(l)}$), we can use state-of-the-art NN verifiers [26] to obtain guaranteed bounds. This approach allows us to construct f_π in a single pass over f with the soundness guarantee given in Theorem 4.1 (see proof on page 17). Below, we describe how errors on the f_π are bounded locally (Section 4.2) and globally (Section 4.3).

Theorem 4.1 (Soundness)**.** *For any neural network f with input space $\mathcal{I}$, the construction approach given in Section 4.1 yields an overflow robust network f_π on $\mathcal{I}$ (see Definition 3.3).*

4.2. Piecewise-Polynomial Overapproximation of Activation Functions

Computing bounds on the univariate approximation error $\pi^{(l)}(x) - \sigma^{(l)}(x)$ is indispensable for our approach and for the Remez algorithm [41] to fit high-quality polynomial approximations. Prior work on verified approximation of $\mathrm{ReLU}(x) = \max(0, x)$ uses its piecewise linearity [14]. This allows bounding the approximation error $\pi(x) - \mathrm{ReLU}(x)$ by finding the extrema of the polynomials $\pi(x) - 0$ for $x \leq 0$ and $\pi(x) - x$ for $x \geq 0$. Finding the extrema of a polynomial $\pi(x)$ in Chebyshev basis reduces to finding the eigenvalues[4]of the Colleague matrix associated with its derivative $\frac{d}{dx}\pi(x)$ [42]. We extend this approach to activation functions that are *monotonically linear almost everywhere*:

Definition 4.1 (Monotonically Linear Almost Everywhere)**.** A function $g : \mathbb{R} \to \mathbb{R}$ is *monotonically linear almost everywhere* iff it has bounded total variation, there are $a_1, a_2, b_1, b_2 \in \mathbb{R}$ with $\lim_{x\to\infty}(g(x) - (a_1x + b_1)) = \lim_{x\to-\infty}(g(x) - (a_2x + b_2)) = 0$ and there are $c_1, c_2 \in \mathbb{R}$ such that $g(x) - (a_1x + b_1)$ (resp. $g(x) - (a_2x + b_2)$) is monotonic on $(-\infty, c_1]$ (resp. $[c_2, \infty)$).

Many activation functions (e.g. GELU, ELU or Sigmoid) satisfy this property (for GELU, we give a proof in Lemma D.1 in the Appendix). We approximate functions of this type via piecewise polynomial approximations of the form

$$q(x) = \mathbb{1}_{x\leq d_0}(x)\cdot(a_1x + b_1) + \mathbb{1}_{x>d_n}(x)\cdot(a_2x + b_2) + \sum_{i=1}^{n} \mathbb{1}_{x\in(d_{i-1},d_i]}(x)\cdot q_i(x)$$

with known approximation error $\epsilon_q \geq |q(x) - \sigma(x)|$ for all $x \in \mathbb{R}$ (with $d_0 \leq c_1, d_n \geq c_2$, and the indicator function $\mathbb{1}_{p(x)}(x) \in \{0, 1\}$ being 1 for x iff $p(x)$). We can then use $q(x)$ as a surrogate in the Remez algorithm to fit a polynomial $\pi^{(l)}(x) \approx q(x) \approx \sigma^{(l)}(x)$ and bound its approximation error

$$|\pi(x) - \sigma(x)| \leq |\pi(x) - q(x)| + |q(x) - \sigma(x)| \leq \epsilon_\pi + \epsilon_q\,, \tag{5}$$

where finding ϵ_π reduces to $n+1$ computations of polynomial extrema and ϵ_q is a known constant.

We can always construct such a function $q(x)$ given a target tolerance ϵ_q. To this end, we first find points $d_0 \leq c_1, d_n \geq c_2$ with $|a_ix + b_i - \sigma(x)| \leq \epsilon_q$ via binary search. Then, we use Chebyshev approximation to find polynomials $q_i(x) \approx \sigma(x)$ over partitions of $[d_0, d_n]$. Given concrete $q_i(x)$, we use interval arithmetic [43] to check if $|q_i(x) - \sigma(x)| \leq \epsilon_q$ over the interval of interest. If the target tolerance is violated, we increase the number of sub-intervals. While this last step is expensive, it only has to be computed *once* for each activation function.

4.3. Differential Bounds via Differential Verification

Construction of f_π according to Section 4.1 guarantees robustness, but we also desire bounds on the deviation $\|f_\pi(\mathbf{x}) - f(\mathbf{x})\|_\infty$ between the polynomial and the original NN. Prior work shows that standard verifiers struggle to tightly bound the difference between two NNs [30], [31], [32]. However, when NNs have the same architecture and similar weights, approaches for differential verification [30], [31], [32], [44] produce tighter bounds by propagating local bounds on the difference through the NN (see Section 2.4.3).

4. This works assumes the correctness of numerically computed eigenvalues.

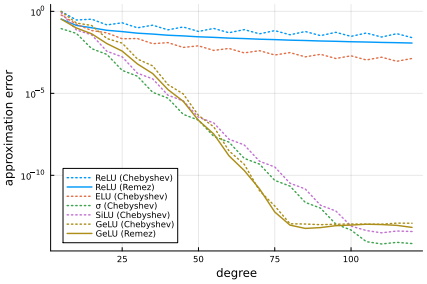


Figure 5: Polynomial approximation error for functions over $x \in [-1, 1]$: Smooth functions converge faster.

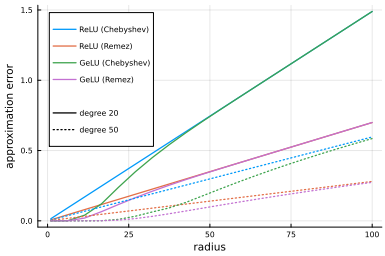


Figure 6: Polynomial approximation error for ReLU and GELU for different intervals of size $[-r, r]$.

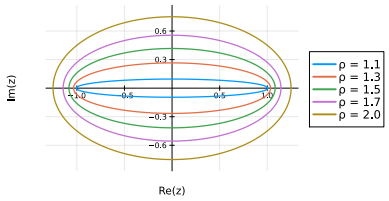


Figure 7: Boundary of Bernstein ellipses E_ρ for different values of parameter ρ.

Differential verification leverages linear over-approximations for activation functions. While existing work provides relaxations for the bivariate non-linearity $\sigma_1^{(l)}(x) - \sigma_2^{(l)}(x - \Delta)$ for $\sigma_1^{(l)} = \sigma_2^{(l)}$ with bounded derivative [44], it cannot handle polynomial $\sigma_1^{(l)}$ for $\sigma_2^{(l)} \neq \mathrm{ReLU}$. Based on the relaxation by Banerjee *et al.* [44], we extend the differential verification tool VERYDIFF [32] with parallel linear relaxations for the case where one NN contains an activation function with bounded derivative and the other contains a polynomial activation (proof see page 18):

Theorem 4.2 (p-σ Relaxation)**.** *Given bounds $x \in [l_x, u_x], y \in [l_y, u_y], \Delta \in [l_\Delta, u_\Delta]$, activation function $\sigma(x)$ with derivative bounds $\partial_l \leq \frac{d}{dx}\sigma(x) \leq \partial_u$ for all $x \in [\min(l_x, l_y), \max(u_x, u_y)]$ and polynomial $p(x)$ with $\epsilon = \max_{x \in [l_x, u_x]} |p(x) - \sigma(x)|$, the activation difference is bounded by*

$$\alpha \cdot \Delta + \beta_l - \epsilon \leq p(x) - \sigma(x - \Delta) \leq \alpha \cdot \Delta + \beta_u + \epsilon \,,$$

where $\alpha = (\alpha_l + \alpha_u)/2$ and

$$\beta_l = \begin{cases} \lambda_l l_\Delta + \hat{\beta}_l & , \lambda_l \geq 0 \\ \lambda_l u_\Delta + \hat{\beta}_l & , \lambda_l < 0 \end{cases}, \; \beta_u = \begin{cases} \lambda_u l_\Delta + \hat{\beta}_u & , \lambda_u \leq 0 \\ \lambda_u u_\Delta + \hat{\beta}_u & , \lambda_u > 0 \end{cases},$$

The $\alpha_l, \alpha_u, \hat{\beta}_l, \hat{\beta}_u$ are the parameters of the non-parallel linear relaxations given by RAVEN *[44] as*

$$(\alpha_l, \alpha_u) = \begin{cases} (\partial_l, \partial_u) & , l_\Delta \geq 0 \\ (\partial_u, \partial_l) & , u_\Delta \leq 0 \\ \left(\frac{\partial_l u_\Delta - \partial_u l_\Delta}{u_\Delta - l_\Delta}, \frac{\partial_u u_\Delta - \partial_l l_\Delta}{u_\Delta - l_\Delta} \right) & , \textit{otw.} \end{cases},$$

$$\lambda_l = (\alpha - \alpha_l) \,, \quad \lambda_u = (\alpha - \alpha_u) \,,$$

$$\hat{\beta}_l = -\hat{\beta}_u = \frac{(\partial_u - \partial_l) l_\Delta u_\Delta}{u_\Delta - l_\Delta} \,.$$

5. Polynomial Approximation and Smoothness

The techniques presented in the previous section enable the certified construction of polynomially approximated networks as well as computing verified bounds on the difference to the original network.These verified bounds can be significantly larger than bounds estimated from a dataset and may thus result in polynomial networks that – although free from overflows – poorly approximate the original networks.

We recover from this loss of precision by choosing networks with smooth activation functions like GELU as the basis for our polynomial networks. Our evaluation (Section 7) will show that this allows our *verified* polynomially approximated networks to eventually achieve smaller approximation error, than polynomial approximation of ReLU networks based on sampled preactivation bounds.

This is based on the fact that the polynomial approximation error of an activation $\sigma : \mathbb{R} \to \mathbb{R}$ converges faster to zero, the smoother the activation function is. The following bounds given by Trefethen *et al.* [42] for the error of degree n Chebyshev polynomial approximations $\pi_n(x) \approx \sigma(x)$ (w.l.o.g. $x \in [-1, 1]$) formally capture this phenomenon:

$$|\sigma(x) - \pi_n(x)| \leq \frac{4V}{\pi\nu(n - \nu)^\nu} \tag{6}$$

$$|\sigma(x) - \pi_n(x)| \leq \frac{4M\rho^{-n}}{\rho - 1} \tag{7}$$

The first bound, implies that the approximation error decays with rate $O(n^{-\nu})$ and is applicable, if σ is ν times differentiable and $n > \nu$. Equation (7) is applicable to activations that are analytic, i.e. their Taylor expansion converges to σ for any $x \in [-1, 1]$, and leads to an even faster decay with rate $O(\rho^{-n})$, where a larger ρ for the Bernstein ellipse E_ρ indicates that σ can be analytically continued further into the complex plane. The constants V and M in the above equations are independent of the polynomial degree n. The former is given by the total variation $V = V_{[-1,1]}(\sigma^\nu)$ of the ν-th derivative of σ, where

$$V_{[a,b]}(\sigma) = \int_a^b |\sigma'(x)| \, dx \tag{8}$$

for continuously differentiable σ (see e.g. [45]). The latter constant $M = \max_{x \in E_\rho} |\sigma(x)|$ is defined using a parameter ρ, s.t. σ can be analytically continued to the interior of the Berstein ellipse

$$E_\rho = \left\{ z \in \mathbb{C} \;\middle|\; z = \frac{\rho e^{i\theta} + \left(\rho e^{i\theta}\right)^{-1}}{2}, \; \rho \geq 1, \; 0 \leq \theta < 2\pi \right\}$$

(definition of E_ρ taken from Wang *et al.* [46]). Bernstein ellipses for varius choices of ρ are illustrated in Figure 7. If σ is *entire* (i.e. it can be analytically continued to all $\mathbb{C}$), then, one can choose any parameter ρ for the Bernstein ellipse that leads to a favourable value of $M = \max_{x \in E_\rho} |\sigma(x)|$ in Equation (7).

The effects of these bounds are visualised in Figure 5: The approximation error for the once-differentiable ELU activation converges faster than the error for the non-differentiable ReLU function. The errors for the analytic functions $\mathrm{Sigm}(x) = 1/(1+e^{-x})$ and $\mathrm{SiLU}(x) = x \cdot \mathrm{Sigm}(x)$ converge much faster. While $\mathrm{Sigm}(x)$ and $\mathrm{SiLU}(x)$ have poles at $x = \pm\pi i$, $\mathrm{GELU}(x)$ can be analytically continued to the entire complex plane. Therefore, we see even faster convergence for this activation function – at least before convergence of the approximation error levels off due to limits of floating point accuracy. Consequently, SiLU and even more so GELU makes precise polynomial approximation of neural networks easier.

Figure 6 illustrates how increases in width of the approximation interval affect the polynomial approximation error, for fixed polynomial degrees. We can see that for smaller radii r of the approximation interval $[-r, r]$, the GELU approximation has a much smaller error than the ReLU approximation, whereas the difference gets smaller as the radius increases. The Figure further shows the benefits of fitting polynomial approximations using the Remez algorithm [41] as opposed to standard Chebyshev interpolation. High-quality polynomial approximations computed by the Remez algorithm are roughly twice as precise as Chebyshev approximations.

6. Concrete CKKS Procedures

The construction of polynomial activations in Section 4.2 implicitly assumes that we can evaluate a different polynomial for each neuron in the same layer l. This goes against standard usages of CKKS-based NNs where typically one evaluates a single polynomial approximation of, e.g. ReLU, over all the neurons in the same layer [10], [12], [25]. However, the per-neuron ranges $[l_i^{(l)}, u_i^{(l)}]$ may differ considerably from each other, and thus much can be gained by approximating them independently, as we demonstrate in Section 7.2.

Here, we show how to evaluate different polynomials in the different slots of a CKKS ciphertexts by simply generalizing the Paterson-Stockmeyer (PS) algorithm. In particular, Chen, Chillotti and Song [16, Theorem 1] first gave a PS version adapted for Chebyshev polynomials. We report, in Algorithm 3, a modified version of the algorithm which works over Chebyshev bases and *sets of coefficients*. In particular, the algorithm proposed in [16] is simply an instantiation of ours when all the sets of coefficients are equal.

We refer to Section 7.1 for further discussion about the CKKS procedures.

7. Experiments

Our experiments[5] aim at evaluating the impact of each contribution on the performance of our approach (Section 7.2), and demonstrate that certified design (with all our improvements) yields comparable accuracy to non-certified design for both plaintext and encrypted inference (Section 7.3).

We run our evaluation on fully connected (FC) NNs trained on the HELOC (Home Equity Line of Credit) [35], MNIST [25] and EC (Energy Consumption) [47] datasets, as well as convolutional (Conv) NNs trained on CIFAR10 [36]. We further retrained FC NNs based on the NN4Sys benchmark [48] for learned index prediction and Conv NNs for remaining useful life prediction (Collins RUL) [49] taken from the annual NN verification competition VNNCOMP [29]. Overall, the NNs cover a wide range of sizes containing between 64 and 10300 neurons. Following prior work on certified design of polynomial NNs based on ReLU NNs [14], we trained all NNs with L_1 regularization to encourage tighter pre-activation bounds.

We use a piecewise-polynomial overapproximation $q(x)$ for the GELU activation function (see Section 4.2) with verified error $|q(x) - \mathrm{GELU}(x)| \leq 10^{-10}$. The overapproximation uses 6 pieces of degree 15 in addition to the linear asymptotes.

Our implementation is written in Julia [50] and builds on top of the differential verifier VERYDIFF [32]. Execution of polynomial NNs under CKKS is based on OpenFHE [17]. To compute verified pre-activation ranges, we leverage the state-of-the-art NN verifier α-CROWN [26]. Experiments for plaintext NNs were run on Ubuntu with a 4-Core Intel Xeon E5-1630v3 and 128 GiB of RAM. CKKS inference was run on a M5 Max CPU with 36 GiB of RAM. Additional information and results for comparison to the state-of-the-art certified construction approach for ReLU neural networks [14] can be found in Appendix C.

7.1. From Certified Networks to CKKS Programs

In order to evaluate our encrypted circuits, we design a compiler that, given a network (defined as a list of linear and Chebyshev layers), generates a JSON file specifying the concrete, certified CKKS circuit. Our OpenFHE implementation takes the JSON file as input and automatically evaluates the layers in the network. We give an overview of our methods in Appendix E. We designed our software to choose the FHE parameters in a way that minimizes the required multiplicative depth and at the same time to support $\lambda > 128$ bits of (classical) security according to [51].

7.2. Ablation Study

We construct polynomial NNs approximating a base NN for multiple values of the polynomial degree d and measure

5. Refer to https://github.com/lorenzorovida/encrypted-neural-networks-without-overflows

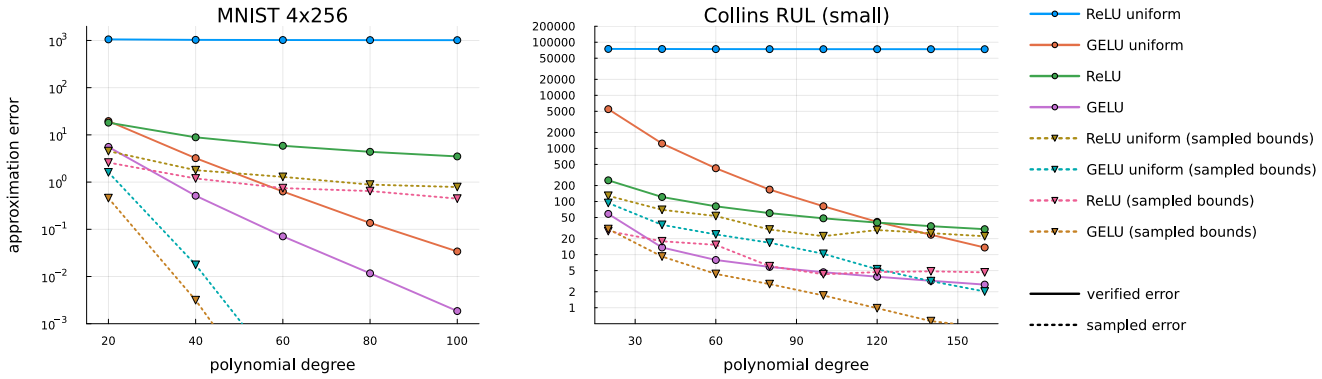


Figure 8: Verified bound on the error $|f_\pi(x) - f(x)|$ for networks with verified pre-activation bounds and error values sampled for networks constructed using sampled bounds. We show results for ReLU and GELU networks with heterogeneous polynomials per layer vs. networks with a single *uniform* polynomial per layer.

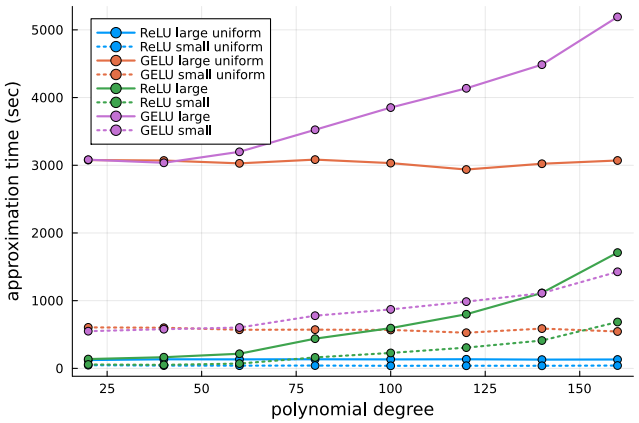


Figure 9: Runtime of certified design for Collins RUL.

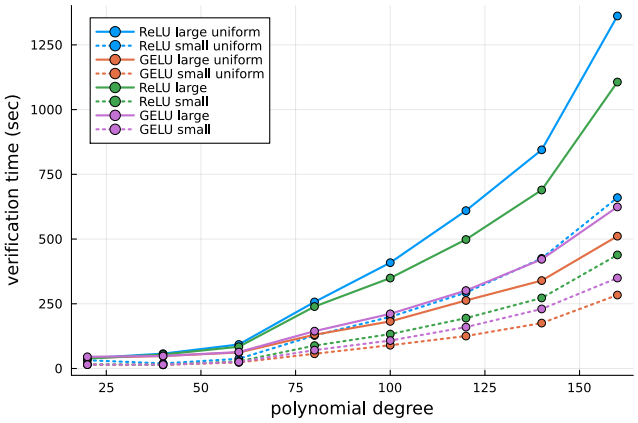


Figure 10: Runtime of verified error bound computation for Collins RUL.

the verifiable bound on the error $\|f_\pi(\mathbf{x}) - f(\mathbf{x})\|_\infty$ as well as the maximum observed error (sampled over the respective dataset). Comparing the error metrics between different construction settings enables us to evaluate the effect of our individual contributions. To show the effects on FC and Conv NNs, we report the results for the MNIST 4×256 NN and the small Collins RUL NN ($[1600, 2400, 800, 600, 100]$ neurons per layer) in Figure 8.

GELU vs. ReLU Networks. Figure 8 clearly shows that the approximation error for GELU NNs is far smaller than for NNs with ReLU activation. This effect gets stronger with increasing polynomial degree d, reflecting the superior approximation properties of the smooth GELU activation (see Section 5). For large degrees and the Collins benchmark, the error values are at least an order of magnitude smaller for GELU NNs than for ReLU NNs, on the MNIST benchmark the improvement is at least three orders of magnitude for $d = 100$. This greater advantage in the MNIST case is likely caused by significantly smaller pre-activation bounds for this benchmark. For larger preactivation bounds, higher polynomial degrees are required to realize the improved convergence of the GELU polynomial approximation (see also Figure 6).

Heterogeneous vs. Uniform Polynomial Layers. Error values when using separately fitted polynomial approximations for each neuron are significantly smaller compared to using a single uniform polynomial approximation for all neurons in the same layer. Exploitation of tighter individual preactivation bounds leads to at least $4.7\times$ smaller approximation error on the Collins benchmark for large degrees. For verified construction of MNIST networks, we achieve even larger improvements of at least $18\times$ for $d = 100$.

Verified vs. Sampled Bounds. Using sampled pre-activation bounds leads to substantially smaller sampled error values. This is expected as sampled values (even if bounds are widened by a constant factor) do not suffer from overapproximation to the same extent as bounds computed using NN verification techniques. However, these polynomial NNs are prone to overflow events. Moreover, the above effect is only valid if we fix the activation function: When we compare the sampled error for ReLU NNs with sampling-based bounds

to the sampling error for GELU NNs with verified bounds, the GELU NNs achieve a better or similar approximation error already for $d \leq 40$ on both MNIST and Collins RUL. The effect is even more pronounced comparing ReLU NNs with a single polynomial per layer (which is the current approach in modern workflows, as in Figure 2) and sampled bounds to verified GELU NNs with multiple polynomials per layer.

7.3. Certified Design at Scale

In this section, we demonstrate that our certified design approach can be applied to networks of varying sizes from different domains. As per Section 7.2, we compute verified pre-activation bounds and fit multiple polynomials per layer. We compare approximations of ReLU and GELU NNs and discuss the quality of the approximated networks as well as the runtime of our approach.

Approximation Quality. For each polynomial network, we measure the verified bound on the approximation error $\|f(\mathbf{x}) - f_\pi(\mathbf{x})\|_\infty$ and cryptographic error over a sample of the respective dataset. We report these values in Table 3 along with the output range of the NNs sampled over the corresponding dataset to put the approximation error into perspective. The table also contains traditional metrics of the NNs: Classification accuracy (HELOC, MNIST, CIFAR), mean squared error (EC, Collins RUL) and mean absolute error (NN4Sys).

Regarding the traditional metrics (classification accuracy, mean squared error, mean absolute error), the results show only very minor drops in performance between the original NN f and its polynomial approximation f_π. The large CIFAR10 NNs with degree 119 being the lone exception.

Although the verified bound on the approximation error $\|f_\pi(x) - f(x)\|_\infty$ increases with network size, it is still fairly tight for most GELU NNs – especially when compared to their typical output ranges. The same is true for the maximum CKKS error $|f_{\text{CKKS}}(x) - f_\pi(x)|$ of the GELU networks. However, polynomial approximations for NNs with ReLU activations, always lead to a much worse verified error bound.

Runtime. We analyze the runtime of our approach for both certified construction of a polynomial NN (see Figure 9), and computation of a verified bound on the approximation error (Figure 10) over a range of polynomial degrees. This experiment was carried out on the Collins RUL benchmark, which contains the largest networks in our benchmark set.

As shown in Figure 9, the runtime of our certified design approach increases for larger polynomial degrees as computation of polynomial roots is more expensive. Synthesis of networks with a single uniform polynomial per layer only requires to run the Remez algorithm once per layer, instead of doing so for every neuron, explaining the large discrepancy in runtime.

Independent of the polynomial degree, there is a large cost related to the runtime of the underlying NN verifier, which scales with the size of the network, but also depends on the computational efficiency of computing relaxations for different activation functions. For AUTOLIRPA [52], the implementation of the α-CROWN NN verification algorithm [26] we use, the relaxations for the GELU activation function are significantly more expensive to compute than the ReLU relaxations.

For computation of the verified bound on the approximation error, we instead use the differential NN verifier VERYDIFF [32] as a backend. In this setting, we observe faster runtimes for the GELU networks, since the relaxation described in Theorem 4.2 for the difference between GELU and its polynomial approximation does not require any computation of polynomial roots – in contrast to the utilized ReLU difference relaxation [14]. Runtime still increases with the polynomial degree, since overapproximation of the individual polynomial activations requires executing the Remez algorithm. More details about how the CKKS experiments have been performed are given in Section 7.1.

8. Conclusions and Directions for Future Work

In this paper, we demonstrate for the first time the vulnerability of current CKKS-based neural networks to overflow events. To solve this, we provide a framework to design certified neural networks. By construction, our approach yields networks that are provably overflow robust on the full input space, without sacrificing predictive performance. Furthermore, our approach can bound the polynomial approximation error of the plaintext via a differential verification procedure. These bounds complement existing estimates on the magnitude of the cryptographic noise e_{CKKS}, thus providing a complete toolkit to assess the maximum output error introduced during CKKS private inference.

Limitations and future work. We see this work as a starting point for more robust FHE inference. Precise modelling of the CKKS noise like in [53], [54] could strengthen our bounds to natively include e_{CKKS} as an additional probabilistic term. Extending our certified design to arbitrary depth networks is possible in theory but requires the inclusion of bootstrapping and co-design of convolution layers [55] to reduce the CKKS noise. Without that, we expect our method to produce slack bounds on deeper architectures, such as those commonly used in state-of-the-art image classifiers, thus reducing its accuracy. Finally, our method may occasionally generate polynomials with ill-scaled coefficients, which cause a reduction on the CKKS precision.

Acknowledgements. The Gemini, Claude, Copilot and Qwen AI models were used for minor editing purposes in this manuscript, such as spellchecking and shortening a few lengthy descriptions. Furthermore, we used them as coding assistants to help with plotting code in Julia and as a starting point in the implementation of the update step (finding points with alternating error) in the Remez algorithm. All outputs were inspected by the authors to ensure accuracy of the final output.

TABLE 3: Verified error bounds and cryptographic error of certified FHE inference. We report the L_1 penalty used for training the original NN, σ is the activation function ($R \equiv \mathrm{ReLU}\,, G \equiv \mathrm{GELU}\,$), N is the ring size, QP is the modulus size. D refers to the multiplicative depth of the circuit. The (max) CKKS error is computed as $|f_{\mathrm{CKKS}}(x) - f_\pi(x)|$ obtained over a sample of the respective dataset. Latencies (t) are in seconds and have been measured on a M5 Max CPU with 36GB of memory. Moduli with * are reported although their security is slightly smaller than 128-bits. Refer to [24, Section 2.3] for the definition of d_{num}

	Size	L_1	σ	Output range (sampled)	Acc, MSE, MAE of f	Poly. deg.	Acc, MSE, MAE of f_π	Verified error	CKKS error	N	QP	q_i	Δ	d_{num}	D	t
HEL.	$[64, 32]$	2e-5	R	$[-5.3, 3.6]$	73.31%	27	73.24%	0.75	10^{-9}	2^{16}	2^{1306}	45	60	2	21	1
			G	$[-4.5, 2.9]$	72.25%	27	72.25%	3.76e-6	10^{-9}	2^{16}	2^{1306}	45	60	2	21	1
MNIST	4×256	1e-4	R	$[-32.0, 19.7]$	98.64%	119	98.64%	2.93	10^{-7}	2^{17}	2^{2746}	45	60	2	49	39
			G	$[-18.9, 25.0]$	98.93%	119	98.93%	3.08e-4	10^{-7}	2^{17}	2^{2746}	45	60	2	49	39
	6×256	1e-4	R	$[-39.4, 25.2]$	99.12%	119	99.09%	9.34	10^{-7}	2^{17}	2^{3841*}	45	60	7	72	74
			G	$[-40.3, 16.7]$	98.94%	119	98.93%	0.046	10^{-7}	2^{17}	2^{3841*}	45	60	7	72	74
CIFAR10	$[900, 392, 144, 256]$	1e-4	R	$[-16.2, 14.7]$	67.24%	119	66.18%	102.47	10^{-4}	2^{17}	2^{3511}	45	60	5	62	95
			R			247	67.57%	49.08	10^{-4}	2^{17}	2^{3511}	45	60	7	66	119
			G	$[-13.9, 17.2]$	69.56%	119	68.70%	26.98	10^{-4}	2^{17}	2^{3511}	45	60	5	62	95
			G			247	69.50%	0.93	10^{-4}	2^{17}	2^{3511}	45	60	7	66	119
	$[1.8k, 784, 288, 256]$	1e-4	R	$[-13.3, 16.4]$	76.30%	119	72.35%	232.48	10^{-4}	2^{17}	2^{3511}	45	60	5	62	167
			R			247	76.31%	111.40	10^{-4}	2^{17}	2^{3511}	45	60	7	66	191
			G	$[-14.7, 20.3]$	77.28%	119	65.20%	85.58	10^{-4}	2^{17}	2^{3511}	45	60	5	62	167
			G			247	75.13%	18.64	10^{-4}	2^{17}	2^{3511}	45	60	7	66	191
EC	1×64	–	R	$[0.2, 2.0]$	0.0839	27	0.0839	0.11	10^{-6}	2^{15}	2^{781}	45	60	2	12	0.5
			G	$[0.2, 2.0]$	0.0844	27	0.0844	3.64e-7	10^{-6}	2^{15}	2^{781}	45	60	2	12	0.5
NN4Sys	3×128	5e-7	R	$[0.0, 1.0]$	4.56e-4	59	4.83e-3	0.089	10^{-9}	2^{17}	2^{2431}	45	60	2	34	6
			G	$[0.0, 1.0]$	3.57e-4	59	3.57e-4	1.75e-6	10^{-9}	2^{17}	2^{2431}	45	60	2	34	6
	5×128	5e-7	R	$[0.0, 1.0]$	5.75e-4	59	3.10e-3	0.11	10^{-9}	2^{17}	2^{3136}	45	60	2	55	22
			G	$[0.0, 1.0]$	2.93e-4	59	2.93e-4	1.15e-5	10^{-9}	2^{17}	2^{3136}	45	60	2	55	22
Collins	$[1.6k, 2.4k, 800, 600, 100]$	1e-3	R	$[0.4, 648.1]$	457.00	119	456.91	40.43	10^{-1}	2^{17}	2^{3511}	45	60	5	61	91
			G	$[1.0, 634.8]$	527.89	119	527.93	3.87	10^{-1}	2^{17}	2^{3511}	45	60	5	61	91
	$[3.2k, 4.8k, 1.6k, 600, 100]$	1e-3	R	$[-8.6, 644.7]$	512.60	119	518.46	68.99	10^{-4}	2^{17}	2^{3511}	45	60	5	61	158
			G	$[0.4, 615.8]$	699.21	119	686.50	9.28	10^{-4}	2^{17}	2^{3511}	45	60	5	61	158

References

[1] B. Pulido-Gaytan, A. Tchernykh, J. M. Cortés-Mendoza, M. Babenko, G. Radchenko, A. Avetisyan, and A. Y. Drozdov, "Privacy-preserving neural networks with homomorphic encryption: C hallenges and opportunities," *Peer-to-Peer Networking and Applications*, vol. 14, no. 3, pp. 1666–1691, 2021.

[2] M. Ribeiro, K. Grolinger, and M. A. Capretz, "Mlaas: Machine learning as a service," in *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, 2015, pp. 896–902.

[3] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ser. ITCS '12, 2012, pp. 309–325.

[4] Z. Brakerski, "Fully homomorphic encryption without modulus switching from classical gapsvp," in *Advances in Cryptology – CRYPTO 2012*, ser. CRYPTO '12, 2012, pp. 868–886.

[5] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," Cryptology ePrint Archive, Paper 2012/144, 2012.

[6] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *Advances in Cryptology – ASIACRYPT 2016*, 2016, pp. 3–33.

[7] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, 2017, pp. 409–437.

[8] J. Lee, E. Lee, J.-W. Lee, Y. Kim, Y.-S. Kim, and J.-S. No, "Precise approximation of convolutional neural networks for homomorphically encrypted data," *IEEE Access*, vol. 11, pp. 62 062–62 076, 2023.

[9] L. Rovida and A. Leporati, "Transformer-based language models and homomorphic encryption: An intersection with bert-tiny," in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, 2024, pp. 3–13.

[10] J. Park, S. Park, J. H. Park, J. H. Ahn, J. H. Cheon, G. Hanrot, J. W. Kim, M. Park, and D. Stehlé, "Scaling up privacy-preserving ML: A CKKS implementation of Llama-2-7B," 2026.

[11] E. Lee, J.-W. Lee, J.-S. No, and Y.-S. Kim, "Minimax approximation of sign function by composite polynomial for homomorphic comparison," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3711–3727, 2022.

[12] A. Ebel, K. Garimella, and B. Reagen, "Orion: A fully homomorphic encryption framework for deep learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025, pp. 734–749.

[13] E. Manino, B. Magri, M. A. Mustafa, and L. C. Cordeiro, "Certified private inference on neural networks via lipschitz-guided abstraction refinement," in *6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS 2023)*, 2023.

[14] P. Kern, E. Manino, and C. Sinz, "Certified error analysis of homomorphically encrypted neural networks," in *AI Verification*. Springer Nature Switzerland, 2026, pp. 156–179.

[15] M. S. Paterson and L. J. Stockmeyer, "On the number of nonscalar multiplications necessary to evaluate polynomials," *SIAM Journal on Computing*, vol. 2, no. 1, pp. 60–66, 1973.

[16] H. Chen, I. Chillotti, and Y. Song, "Improved bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2019*, 2019, pp. 34–54.

[17] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, "Openfhe: Open-source fully homomorphic encryption library," in *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, ser. WAHC'22, 2022, p. 53–63.

[18] B. Li, D. Micciancio, M. Schultz-Wu, and J. Sorrell, "Securing approximate homomorphic encryption using differential privacy," in *Advances in Cryptology – CRYPTO 2022*, 2022.

[19] A. Alexandru, A. A. Badawi, D. Micciancio, and Y. Polyakov, "Application-aware approximate homomorphic encryption: Configuring FHE for practical use," *IACR Communications in Cryptology*, vol. 2, no. 4, 2026.

[20] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Advances in Cryptology – EUROCRYPT 2010*, 2010, pp. 1–23.

[21] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, Sep. 2009.

[22] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, 2018, pp. 360–384.

[23] C. Gentry, S. Halevi, and N. P. Smart, "Better bootstrapping in fully homomorphic encryption," in *Public Key Cryptography – PKC 2012*, 2012, pp. 1–16.

[24] A. Kim, A. Papadimitriou, and Y. Polyakov, "Approximate homomorphic encryption with reduced approximation error," in *Topics in Cryptology – CT-RSA 2022: Cryptographers' Track at the RSA Conference 2022*, 2022, pp. 120–144.

[25] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.

[26] K. Xu, H. Zhang, S. Wang, Y. Wang, S. Jana, X. Lin, and C. Hsieh, "Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers," in *Proc. of the 9th International Conference on Learning Representations(ICLR'21), Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.

[27] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljić, D. L. Dill, M. J. Kochenderfer, and C. Barrett, "The Marabou Framework for Verification and Analysis of Deep Neural Networks," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 11561, 2019, pp. 443–452.

[28] P. Henriksen and A. Lomuscio, "Efficient neural network verification via adaptive refinement and adversarial search," in *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2020, vol. 325, pp. 2513–2520.

[29] K. Kaulen, T. Ladner, S. Bak, C. Brix, H. Duong, T. Flinkow, T. T. Johnson, L. Koller, E. Manino, T. H. Nguyen, and H. Wu, "The 6th international verification of neural networks competition (VNN-COMP 2025): Summary and results," *CoRR*, vol. abs/2512.19007, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2512.19007

[30] B. Paulsen, J. Wang, and C. Wang, "ReluDiff: differential verification of deep neural networks," in *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, G. Rothermel and D. Bae, Eds. ACM, 2020, pp. 714–726.

[31] B. Paulsen, J. Wang, J. Wang, and C. Wang, "NeuroDiff: scalable differential verification of neural networks using fine-grained approximation," in *35th IEEE/ACM International Conference on Automated Software Engineering,*, 2020, pp. 784–796.

[32] S. Teuber, P. Kern, M. Janzen, and B. Beckert, "Revisiting differential verification: Equivalence verification with confidence," in *Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS*, 2025, pp. 257–278.

[33] I. Nam, "The avg-act swap and plaintext overflow detection in fully homomorphic operations over deep circuits," in *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '24, 2024, p. 127–138.

[34] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations, ICLR 2018,*, 2018.

[35] FICO. (2018) Fico explainable machine learning challenge. [Online]. Available: https://community.fico.com/s/explainable-machinelearning-challenge

[36] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Toronto, Ontario, Tech. Rep. 0, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[37] Y. Bengio, N. Léonard, and A. C. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *CoRR*, vol. abs/1308.3432, 2013.

[38] J. J. Hull, "A database for handwritten text recognition research," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 5, pp. 550–554, 1994. [Online]. Available: https://doi.org/10.1109/34.291440

[39] A. Torralba, R. Fergus, and W. T. Freeman, "80 million tiny images: A large data set for nonparametric object and scene recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 11, pp. 1958–1970, 2008.

[40] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do cifar-10 classifiers generalize to cifar-10?" 2018, https://arxiv.org/abs/1806.00451.

[41] E. Remes, "Sur un procédé convergent d'approximations successives pour déterminer les polynômes d'approximation," *CR Acad. Sci. Paris*, vol. 198, pp. 2063–2065, 1934.

[42] L. N. Trefethen, *Approximation Theory and Approximation Practice*. SIAM, 2019. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9781611975949

[43] R. E. Moore, *Interval Analysis*. Englewood Cliffs, NJ: Prentice-Hall, 1966.

[44] D. Banerjee, C. Xu, and G. Singh, "Input-relational verification of deep neural networks," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, pp. 1–27, 2024. [Online]. Available: https://doi.org/10.1145/3656377

[45] J. Appell, J. Banas, and N. J. M. Díaz, *Bounded Variation and Around*. Berlin, Boston: De Gruyter, 2014. [Online]. Available: https://doi.org/10.1515/9783110265118

[46] H. Wang and L. Zhang, "Jacobi polynomials on the bernstein ellipse," *J. Sci. Comput.*, vol. 75, no. 1, p. 457–477, Apr. 2018. [Online]. Available: https://doi.org/10.1007/s10915-017-0542-4

[47] C. for Energy Regulation, "Electricity smart metering customer behaviour trials (cbt) findings report," 2011.

[48] S. Lin, H. He, T. Wei, K. Xu, H. Zhang, G. Singh, C. Liu, and C. Tan, "Nn4sysbench: Characterizing neural network verification for computer systems," in *Advances in Neural Information Processing Systems*, 2024.

[49] D. Kirov, S. F. Rollini, L. D. Guglielmo, and D. D. Cofer, "Formal verification of a neural network based prognostics system for aircraft equipment," in *Bridging the Gap Between AI and Reality - First International Conference, AISoLA 2023, Proceedings*. Springer, 2023, pp. 225–240.

[50] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM review*, vol. 59, no. 1, pp. 65–98, 2017. [Online]. Available: https://doi.org/10.1137/141000671

[51] J.-P. Bossuat, R. Cammarota, I. Chillotti, B. R. Curtis, W. Dai, H. Gong, E. Hales, D. Kim, B. Kumara, C. Lee, X. Lu, C. Maple, A. Pedrouzo-Ulloa, R. Player, Y. Polyakov, L. A. R. Lopez, Y. Song, and D. Yhee, "Security guidelines for implementing homomorphic encryption," *IACR Communications in Cryptology*, vol. 1, no. 4, 2025.

[52] K. Xu, Z. Shi, H. Zhang, Y. Wang, K. Chang, M. Huang, B. Kailkhura, X. Lin, and C. Hsieh, "Automatic perturbation analysis for scalable certified robustness and beyond," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems (NeurIPS'20), December 6-12, 2020, virtual*, 2020.

[53] A. Costache, B. R. Curtis, E. Hales, S. Murphy, T. Ogilvie, and R. Player, "On the precision loss in approximate homomorphic encryption," in *Selected Areas in Cryptography – SAC 2023: 30th International Conference, Fredericton, Canada, August 14–18, 2023, Revised Selected Papers*, 2023, pp. 325–345.

[54] J.-P. Bossuat, A. Costache, C. Mouchet, L. Nürnberger, and J. R. Troncoso-Pastoriza, "Accurate and composable noise estimates for CKKS with application to exact HE computation," *IACR Communications in Cryptology*, vol. 2, no. 2, 2025.

[55] J. H. Ju, J. Park, J. Kim, M. Kang, D. Kim, J. H. Cheon, and J. H. Ahn, "Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping," in *Proceedings of ACM CCS*, 2024, pp. 4361–4375.

[56] Z. Shi, Q. Jin, Z. Kolter, S. Jana, C. Hsieh, and H. Zhang, "Neural network verification with branch-and-bound for general nonlinearities," in *Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS*, 2025, pp. 315–335.

[57] Y. Biktairov and J. Deshmukh, "SOL: sampling-based optimal linear bounding of arbitrary scalar functions," in *Advances in Neural Information Processing Systems*, 2023.

[58] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[59] L. Rovida and A. Leporati, "Encrypted image classification with low memory footprint using fully homomorphic encryption," *International Journal of Neural Systems*, vol. 34, no. 05, p. 2450025, 2024.

[60] J. W. Bos, W. Castryck, I. Iliashenko, and F. Vercauteren, "Privacy-friendly forecasting for the smart grid using homomorphic encryption and the group method of data handling," in *Progress in Cryptology - AFRICACRYPT 2017*, 2017, pp. 184–201.

[61] A. Veit, C. Goebel, R. Tidke, C. Doblander, and H.-A. Jacobsen, "Household electricity demand forecasting: benchmarking state-of-the-art methods," in *Proceedings of the 5th International Conference on Future Energy Systems*, ser. e-Energy '14, 2014, p. 233–234.

[62] R. Dathathri, B. Kostova, O. Saarikivi, W. Dai, K. Laine, and M. Musuvathi, "Eva: an encrypted vector arithmetic language and compiler for efficient homomorphic computation," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020, 2020, p. 546–561.

[63] Y. Lee, S. Heo, S. Cheon, S. Jeong, C. Kim, E. Kim, D. Lee, and H. Kim, "Hecate: Performance-aware scale optimization for homomorphic encryption compiler," in *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022, pp. 193–204.

Appendix A. Improved GELU Relaxation

In an attempt to compute tighter bounds for Equation (2) for GELU activations, we tried to improve upon the parameterized GELU relaxation given by Shi *et al.* [56] used in α-CROWN [26]. While our relaxation achieves bounds that are significantly tighter at parameter initialization, the difference to the relaxation by Shi *et al.* after parameter optimization is minimal. Since we used our relaxation in our code, we nevertheless give a brief overview over its construction.

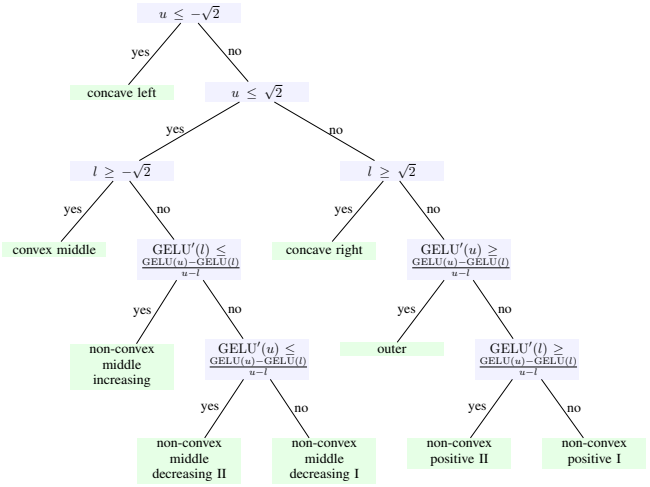


Figure 11: Case distinction for improved GELU relaxation.

At a high level, we provide linear over- and underapproximations

$$\underline{a}(\alpha_1, l, u)x + \underline{b}(\alpha_1, l, u) \leq \mathrm{GELU}(x) \tag{9}$$

$$\leq \overline{a}(\alpha_2, \alpha_3, l, u) + \overline{b}(\alpha_2, \alpha_3, l, u) \quad \forall x \in [l, u]\ , \tag{10}$$

where our relaxation is parameterized by a lower tangent point α_1, an upper left tangent point α_2 and an upper right tangent point α_3. We then use the case distinction shown in Figure 11 to select if one of the tangents or a secant line is a valid lower or upper bound.

The GELU function is concave for $x \in (-\infty, -\sqrt{2}]$ and $x \in [\sqrt{2}, \infty)$. This relates to the cases *concave left* and *concave right* in Figure 11. If $[l, u]$ is completely within these regions, the secant is always the best linear lower bound and the tangent at any point $x \in [l, u]$ is a valid linear upper bound. For initialization, we set the tangent point (either α_2 or α_3) to $t = 1/2 \cdot (l + u)$. This choice leads to an area-optimal overapproximation [57].

Since GELU is convex for $x \in [-\sqrt{2}, \sqrt{2}]$, we can use the secant as an upper bound and set α_1 similarly as above.

In the cases *non-convex middle increasing* and *outer*, we can use the secant as a valid upper bound and the tangent at $t \in [l', u']$ for some $l \leq l', u' \leq u$ as a valid lower bound. We employ the same procedure as Shi *et al.* [56] for finding l', u' and clamp the value of α_1 to these bounds if necessary.

For the cases *middle decreasing II* and *non-convex positive II*, the secant is a valid lower bound and the tangent at $t \in [l, u']$, respectively the tangent at $t \in [l', u]$ is a valid upper bound. In these cases, we initialize α_2 or α_3 by computing $1/2 \cdot (l + u)$ and clamping to the valid bounds.

Finally, for *middle decreasing I* and *non-convex positive I*, the tangent at $t \in [l', u']$ is a valid lower bound and the

tangent at $t \in [l, u'']$ or respectively $t \in [l'', u]$ is a valid upper bound. As initialization for α_1, α_2 and α_3, we choose again the mid-point of the interval $[l, u]$ clamped to the valid tangent point ranges.

Appendix B. Additional Information on Differential Verification for Neural Networks

In this section, we provide information for how to instantiate the differential relaxation presented in Theorem 4.2 for the GELU function and report an optimization used to tighten the verified differential bounds.

B.1. Bounds on the Derivative of GELU

Instantiating the differential relaxation for $p(x) - \sigma(x - \Delta)$ as described in Theorem 4.2 for $\sigma = \mathrm{GELU}$, requires computing bounds on the derivative $\mathrm{GELU}'(x)$ of the GELU function over an interval $x \in [l, u]$.

Since $\mathrm{GELU}(x)$ is concave for $D_1 = (-\infty, -\sqrt{2}]$, convex for $D_2 = [-\sqrt{2}, \sqrt{2}]$ and concave for $D_3 = [\sqrt{2}, \infty)$, its derivative is monotonically decreasing, monotonically increasing and monotonically decreasing over the respective domains. Therefore, the extrema can only be attained at the boundaries of these regions:

$$C = \bigcup_{i \in \{1,2,3\}} \{x \mid x \in \{l_i, u_i\},\ [l_i, u_i] = D_i \cap [l, u]\} \ ,$$

where we do not add the interval boundaries of degenerate intervals. We can then just optimize over this finite set and use

$$\min / \max \left\{ \mathrm{GELU}'(x) \mid x \in C \right\} \ . \tag{11}$$

B.2. Increased Precision using Construction-time Verified Ranges

Note that the formulas to compute the neuron-wise differences $\tilde{\boldsymbol{\Delta}}^{(l)}$ and $\boldsymbol{\Delta}^{(l)}$ in Section 2.4.2 still partially depend on the values $\mathbf{y}_i^{(l-1)}$ and $\tilde{\mathbf{y}}_i^{(l)}$ of neurons in the individual networks $\mathcal{N}_1$ and $\mathcal{N}_2$. Our differential verification approach extends VERYDIFF [32], which computes neuron ranges based on zonotope propagation and is thus limited to linear relaxations with parallel slopes. State-of-the-art NN verifiers like α-CROWN [26] are known to compute tighter bounds for ranges in single neural networks. If such a NN verifier was used to compute the verified pre-activation bounds of the polynomial network, we can use the tightened lower and upper bounds

$$l = \max\{l_{zono}, l_p\}, \qquad u = \min\{u_{zono}, u_p\} \ , \tag{12}$$

where $[l_{zono}, u_{zono}]$ are the ranges computed by VERYDIFF and $[l_p, u_p]$ are the verified ranges for the polynomial network.

Appendix C. Additional Evaluation Results

In this appendix, we provide a more detailed description of our benchmarks and base neural networks. Furthermore, we present additional evaluation results.

C.1. Benchmarks

In the following, we give a brief description of each dataset and the associated machine learning task. As prior work [14] has already shown that training the base networks with L_1-regularization significantly reduces the polynomial approximation error, we trained all NNs with L_1-penalty (see Table 3).

We run our approach on benchmarks related to classification tasks to evaluate the difference in accuracy between base and polynomial network, and also evaluate on regression tasks, where the verified error bounds are a more meaningful measure for comparison between the base and the polynomial network.

C.1.1. Classification Benchmarks.

HELOC. The *home equity line of credit* dataset [35] contains 23 features used to predict if a person is creditworthy. Following the procedure of prior work on certified construction of polynomial networks [14], we normalized the features to $[0, 1]$. We also use the ReLU neural networks provided by them. Neural networks with GELU activation are obtained by training directly on the HELOC dataset.

MNIST. Neural networks trained on the MNIST dataset [58] aim to correctly classify the handwritten digit shown on 28×28 pixel grayscale images. Pixel values are normalized to $[0, 1]$. Since Kern *et al.* [14] also evaluated on MNIST, we simply reuse their ReLU networks. Additionally, we train networks of the same architecture with GELU activation on the MNIST dataset to obtain base networks with this activation function.

CIFAR10. The task associated with the CIFAR10 dataset [36] is to assign the correct labels to images represented by inputs of dimension $3 \times 32 \times 32$, where each dimension can attain values in $[0, 1]$. The dataset is a popular benchmark used to evaluate FHE neural networks [12], [59]. However, those approaches rely on sampled pre-activation bounds. To achieve good accuracy of the polynomial networks – despite wide verified bounds due to overapproximation incurred for large networks by NN verifiers – we train two convolutional networks of different sizes ourselves. The difference in performance between the small and the large network can then be used to judge scalability of our verified approach.

C.1.2. Regression Benchmarks.

Residential Electricity Forecasting (EC). Prior research on FHE used electricity demand forecasting as a use case [60]. In such setting, the smart meter associated to a residential building would send encrypted consumption data to a centralised server and get a short-term (half-hour) prediction

of the future load. Similar to [60], we train a network that receives the past 24 hours of consumption data as input (averaged across 10 homes), and predicts the next half-hour consumption. Additional input features include day of the year, day of the week, and time of the day. The dataset is real-world consumption data from a large-scale study from 2011, Ireland [47]. As discovered by [61], a small fully-connected network with a single hidden layer suffice to maximise predictive accuracy; we choose a width of 64 neurons.

NN4Sys. We use the *learned index* models from the NN4Sys benchmark [48] used in the annual neural network verification competition VNNCOMP [29]. The associated task is, to map a key (a scalar in $[0, 1]$) to an index (also a scalar in $[0, 1]$) in a database. Since the available networks are not trained using L_1 regularization, we extract a training dataset from the input-output specifications that were used in the competition to train $relu$ and $gelu$ neural networks from scratch. We use the same architecture as the non-L_1 regularized networks in the original NN4Sys benchmark.

Collins RUL. The Collins RUL benchmark [49] was also used in the competition VNNCOMP [29]. It consists of convolutional networks that process multivariate time series data (windows of 20 time steps for 20 sensors) to predict the remaining useful life of aircraft components. Since the original networks provided by the benchmark are, again, not trained using L_1 regularization, we train ReLU and GELU networks of the same structure ourselves. However, as the benchmark is only accompanied by a small test dataset, we train our networks to minimize the mean squared error $(\mathcal{N}_{L_1}(x) - \mathcal{N}(x))^2$ to the original network over random data (while monitoring the loss over the test dataset).

C.2. Comparison to Prior Certified Approaches

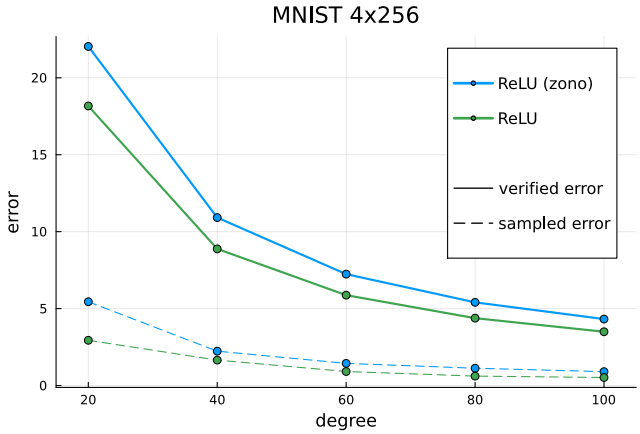


Figure 12: Comparison of our method to ZONOPOLY [14] for verified bounds and sampled values on $\|f_\pi(x) - f(x)\|_\infty$.

We also conducted experiments to compare our approach to ZONOPOLY [14] the prior state of the art in certified construction of polynomial neural networks.

Both our approach and ZONOPOLY extend the zonotope-based differential verification tool VERYDIFF [32] to compute bounds on the output difference between original and polynomial neural networks. However, our approach can leverage state of the art neural network verifiers (e.g. [26]) to compute bounds on the networks' preactivation ranges (see Section 4.1), while ZONOPOLY is restricted to less powerful zonotope-based methods for this step.

As ZONOPOLY is only applicable to fully connected networks with ReLU activation function, we only compare performance on the MNIST 256×4 network with ReLU activation. Starting from this base network, we constructed polynomial networks for a range of polynomial degrees and measured the verifiable bounds on the error $\|f_\pi(x) - f(x)\|_\infty$, as well as the sampled error over the MNIST dataset for each method.

The results shown in Figure 12 demonstrate that our approach achieves consistently better error bounds than ZONOPOLY ($1.21\times$ and $1.23\times$ smaller verified error).

Appendix D. Proofs

Proof of Theorem 4.1. A neural network f_π is overflow robust iff for all layers l it holds that its polynomials $\pi^{(l)}$ are designed for an interval $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ such that for all $\mathbf{x} \in \mathcal{I}$ it holds that $f_\pi^{(l)}(\mathbf{x}) \in [\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$.

Note that our approach computes the intervals $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ by solving the maximization problem in equation (2) with the modification given in (3). To prove that our approach generates certified neural networks f_π we must thus prove that at the time of polynomial construction the optimization problem in equations (2) and (3) correctly over-approximates the behaviour of $f_\pi^{(l)}$. Before we do so below, observe that this property suffices to guarantee certified network construction: Every time an activation function is substituted by its polynomial approximation, we then compute valid bounds for $f_\pi^{(l)}$ using equations (2) and (3) and construct an appropriate polynomial using the Remez algorithm for the sound range estimate.

Formally, proving that equations (2) and (3) correctly over-approximates the behaviour of $f_\pi^{(l)}$ amounts to showing that for any $\mathbf{x} \in \mathcal{I}$ and any $1 \leq l \leq L$ it holds that there exist assignments to the optimization problem such that $\tilde{\mathbf{y}}^{(l)} = f_\pi^{(l)}(\mathbf{x})$.

We prove the latter inductively: First, observe that for $l = 1$ the equations *exactly* represent the behaviour of $f_\pi^{(1)}$. As $f_\pi^{(1)} = f^{(1)}$, $\tilde{\mathbf{y}}^{(l)}$ and $f_\pi^{(l)}(\mathbf{x})$ naturally coincide for all $\mathbf{x} \in \mathcal{I}$.

For our induction step $(l + 1)$, we assume that we have shown the over-approximation property for layer l. Due to this, we know that the Remez algorithm computes a polynomial approximation $\pi^{(l)}$ for $\sigma^{(l)}$ on the sound range $[\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$ for which we know that for all $\mathbf{x} \in \mathcal{I}$ we have $f^{(l)}(\mathbf{x}) \in [\mathbf{l}^{(l)}, \mathbf{u}^{(l)}]$. Since we assume correct local bound computation (see Section 4.2 for detailed approach), we also

know that $\left|\pi^{(l)}\left(f^{(l)}(\mathbf{x})\right) - \sigma^{(l)}\left(f^{(l)}(\mathbf{x})\right)\right| \leq \boldsymbol{\epsilon}^{(l)}$ for any $\mathbf{x} \in \mathcal{I}$.

For any $\mathbf{z}^{(0)} = \mathbf{x} \in \mathcal{I}$, we now consider some concrete assignment of the variables in equations (2) and (3) such that $\tilde{\mathbf{z}}^{(l)} = f^{(l)}(\mathbf{x})$ (exists due to induction hypothesis). We can then set $\boldsymbol{\delta}^{(l)}$ to $\pi^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right) - \sigma^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right) = \pi^{(l)}\left(f^{(l)}(\mathbf{x})\right) - \sigma^{(l)}\left(f^{(l)}(\mathbf{x})\right)$ which we know to satisfy the constraint $\boldsymbol{\delta}^{(l)} \in [-\boldsymbol{\epsilon}^{(l)}, \boldsymbol{\epsilon}^{(l)}]$. Consequently, we can assign $\mathbf{z}^{(l)}$ to $\pi^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right)$ (as $\sigma^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right) + \left(\pi^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right) - \sigma^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right)\right) = \pi^{(l)}\left(\tilde{\mathbf{z}}^{(l)}\right)$). Finally, we assign $\tilde{\mathbf{z}}^{(l+1)}$ to $W^{(l+1)}\mathbf{z}^{(l)} + \mathbf{b}^{(l+1)}$. First, note that both of these assignments satisfy all constraints of equations (2) and (3). Second, observe that by definition of f_π, if $\tilde{\mathbf{z}}^{(l)} = f^{(l)}(\mathbf{x})$ then now it also holds that $\tilde{\mathbf{z}}^{(l+1)} = f^{(l+1)}(\mathbf{x})$. This concludes our induction step of the over-approximation property. □

If a function satisfies all requirements of Definition 4.1 except for the monotonicity requirement, we call it linear almost everywhere (LAE). We can then derive the following result which enables us to approximate its outermost regions with the linear functions representing the asymptotes:

Proposition D.1 (Arbitrarily Precise Linear Approximation)**.** *If a function is LAE, then for any ϵ^* there exists a $\delta < 0$ (resp. $\delta > 0$) such that $|g(x) - (a_i x + b_i)| \leq \epsilon^*$ for all $x \in (-\infty, \delta]$ for $i = 1$ (resp. for all $x \in [\delta, \infty)$ for $i = 2$).*

Proof. This result follows directly from the definition of function limits which states $g(x) - a_i x + b_i$ has limit 0 iff the following formula is valid:

$$\forall \epsilon > 0\ \exists c > 0\ \forall x\ \ x > c \rightarrow |(g(x) - a_i x + b_i) - 0| < \epsilon.$$

For any given ϵ^* we thus choose δ as the c from the formula above. □

Lemma D.1 (GeLU is MLAE)**.** *The function* $\mathrm{GELU}(x) = x \cdot \boldsymbol{\Phi}(x)$ *(for $\boldsymbol{\Phi}(x)$ being the gaussian error function) is monotonically linear almost everywhere.*

Proof. We know that $0 \leq \boldsymbol{\Phi}(x) \leq 1$ and therefore $0 \leq 1 - \boldsymbol{\Phi}(x) \leq 1$.

Additionally, it is easily derivable that:

$$\begin{aligned} 1 - \boldsymbol{\Phi}(x) &= \int_x^\infty \phi(u)\, du = \int_x^\infty \frac{u}{u}\phi(u)\, du \\ &\leq \int_x^\infty \frac{u}{x}\phi(u)\, du = \frac{1}{x}\int_x^\infty u\phi(u)\, du \end{aligned}$$

Since $\phi'(x) = -x \cdot \phi(x)$, the last term can be simplified to $\frac{1}{x}\left[-\phi(u)\right]_x^\infty$ which yields $\frac{\phi(x)}{x}$ as ϕ's limit is 0 for $x \to \pm\infty$.

We then know that for $x < 0$ it holds that $|\mathrm{GELU}(x)| = |x \cdot \boldsymbol{\Phi}(x)| = (-x) \cdot (1 - \boldsymbol{\Phi}(-x)) \leq (-x)\frac{\phi(-x)}{-x} = \phi(-x) \to 0$ (for $x \to -\infty$). Further, since $\mathrm{GELU}(x) < 0$ for $x < 0$, this yields the limit for $-\infty$ via squeezing.

We can also show that $\mathrm{GELU}(x) - x = x(\boldsymbol{\Phi}(x) - 1) = -x(1 - \boldsymbol{\Phi}(x)) \geq (-x)\frac{\phi(x)}{x} = -\phi(x)$. Since $-x(1 - \boldsymbol{\Phi}(x)) \leq 0$ for $x \geq 0$ and $-\phi(x) \to 0$ (for $x \to \infty$) this yields that $\lim_{x\to\infty} \mathrm{GELU}(x) - x = 0$ via squeezing.

Also, let $0 > x^* = \arg\min_{x\in\mathbb{R}} \mathrm{GELU}(x)$, then for $x \in (-\infty, x^*]$, $\mathrm{GELU}(x)$ is monotonically decreasing.

Additionally, we have

$$\begin{aligned} \mathrm{GELU}(-x) &= (-x) \cdot \Phi(-x) \\ &= (-x) \cdot (1 - \Phi(x)) = \mathrm{GELU}(x) - x\ . \end{aligned}$$

Therefore, $\mathrm{GELU}(x) \leq x$ for $x \in [0, \infty)$ and $\mathrm{GELU}(x) - x$ is monotonically decreasing for $x \in [-x^*, \infty)$. □

Soundness of our linear relaxation depends on the relaxation for activation functions with bounded derivative given by RAVEN [44, Appendix G.2]. We extend the relaxation introduced by Banerjee *et al.* [44] to a differential transformer for $p(x) - \sigma(x - \Delta)$ for polynomial p and activation function σ using a verified bound on the approximation error $\epsilon \geq \max_{x\in[l,u]} |p(x) - \sigma(x)|$. Given this, we can overapproximate the difference $\delta = p(x) - \sigma(x - \Delta)$ by

$$\sigma(x) - \sigma(x - \Delta) - \epsilon \leq \delta \leq \sigma(x) - \sigma(x - \Delta) + \epsilon \quad (13)$$

and if σ has bounded derivative, we can therefore apply the corresponding relaxation for $f = g = \sigma$ given in RAVEN [44].

However, since VERYDIFF [32] requires parallel linear relaxations, we have to slightly adjust the non-parallel RAVEN-relaxation.

Proof of Theorem 4.2. We construct parallel lower and upper linear relaxations for $\sigma(x) - \sigma(x - \Delta)$ by slightly modifying the non-parallel RAVEN relaxation RAVEN [44, Appendix G.2]:

$$\alpha_l \cdot \Delta + \hat{\beta}_l \leq \sigma(x) - \sigma(x - \Delta) \leq \alpha_u \cdot \Delta + \hat{\beta}_u\ . \quad (14)$$

We first compute the mean slope

$$\alpha = \frac{1}{2}(\alpha_l + \alpha_u) \quad (15)$$

of the non-parallel linear lower and upper relaxations. Then we compute upward and downward shifts for the linear function $\alpha \cdot \Delta$ s.t. it is larger than RAVEN's upper relaxation $\alpha_u \cdot \Delta + \beta_u$ and smaller than RAVEN's lower relaxation $\alpha_l \cdot \Delta + \beta_l$. Finally, we apply Equation 13 to lift the correctness result from $\sigma(x) - \sigma(x - \Delta)$ to a result for $p(x) - \sigma(x - \Delta)$.

For the upward shift, we compute

$$\begin{aligned} \beta_u &= \max_{\Delta\in[l_\Delta, u_\Delta]} \alpha\Delta - (\alpha_u\Delta + \hat{\beta}_u) \\ &= \max_{\Delta\in[l_\Delta, u_\Delta]} (\alpha - \alpha_u)\Delta + \hat{\beta}_u \\ &= \begin{cases} (\alpha - \alpha_u)l_\Delta + \hat{\beta}_u & , \alpha - \alpha_u \leq 0 \\ (\alpha - \alpha_u)u_\Delta + \hat{\beta}_u & , \alpha - \alpha_u > 0\ , \end{cases} \end{aligned}$$

where the case distinction is due to changing monotonicity of $(\alpha - \alpha_u)\Delta + \hat{\beta}_u$.

Similarly, we compute the shift for the lower bound as

$$\begin{aligned}\beta_l &= \min_{\Delta\in[l_\Delta,u_\Delta]} \alpha\Delta - (\alpha_l\Delta + \hat{\beta}_l) \\ &= \min_{\Delta\in[l_\Delta,u_\Delta]} (\alpha-\alpha_l)\Delta + \hat{\beta}_l \\ &= \begin{cases} (\alpha-\alpha_l)l_\Delta + \hat{\beta}_l & ,\alpha-\alpha_l \geq 0 \\ (\alpha-\alpha_l)u_\Delta + \hat{\beta}_l & ,\alpha-\alpha_l < 0 \end{cases}\end{aligned}$$

With the above steps, we obtain

$$\alpha\Delta + \beta_l \leq \sigma(x) - \sigma(x-\Delta) \leq \alpha\Delta + \beta_u \tag{16}$$

over the domain of interest. Applying Equation 13, we obtain our final statement

$$\alpha\Delta + \beta_l - \epsilon \leq p(x) - \sigma(x-\Delta) \leq \alpha\Delta + \beta_u + \epsilon\,. \tag{17}$$

□

Appendix E. Network to JSON grammar

Our goal is to, given a polynomial network, being able to evaluate it under CKKS. In the literature we can find different examples of general purpose HE compilers, such as EVA [62] or HECATE [63]. Unlike these frameworks, our input to the CKKS program is a JSON file containing a list of NN layers, with corresponding sizes and weights, as defined in the grammar in Figure 13. Our goal is not to be as generic as the previously mentioned compilers. Instead, we tailor the CKKS circuit construction to the NN use case and implemented an automatic FHE evaluation of each specified layer. Moreover, our evaluation takes as input coefficients of Chebyshev polynomials, one for each neuron, which enables the evaluation of overflow-free networks under CKKS using Algorithm 3. This evaluation of heterogenuous polynomials in a single layer is not, to the best of our knowledge, possible in any HE compiler.

E.0.1. Automatic evaluation of linear layers. We implemented simple algorithms to evaluate linear layers. We first define the following encodings.

Definition E.1 (Repeated encoding)**.** Given a vector $\boldsymbol{v} \in \mathbb{R}^n$, its *repeated encoding* is a vector $\boldsymbol{v}' \in \mathbb{R}^{n^2}$ where $\boldsymbol{v}$ is repeated n times. Visually:

$$\boldsymbol{v}' := (v_1, v_2, \ldots, v_n \,|\, v_1, v_2, \ldots, v_n \,|\, \ldots) \in \mathbb{R}^{n^2}.$$

Definition E.2 (Expanded encoding)**.** Given a vector $\boldsymbol{v} \in \mathbb{R}^n$, its *expanded encoding* is a vector $\boldsymbol{v}' \in \mathbb{R}^{n^2}$ where each component v_i of $\boldsymbol{v}$ is repeated n times. Visually:

$$\boldsymbol{v}' := (v_1, v_1, \ldots, v_1 \,|\, v_2, v_2, \ldots, v_2 \,|\, \ldots) \in \mathbb{R}^{n^2}.$$

We use two different input encodings as we employ two different algorithms to evaluate the typical $W\boldsymbol{x} + \boldsymbol{b}$ transformation, depending on how the input is encoded (Algorithm 1 and 2).

Algorithm 1 Linear layer with expanded input

Input: Expanded ciphertext $\boldsymbol{c} \in \mathbb{R}^{n^2}$, column-wise weight matrix $\boldsymbol{w} \in \mathbb{R}^{n^2}$, repeated bias $\boldsymbol{b} \in \mathbb{R}^{n^2}$, dimension n
Output: Repeated encoding of $W\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^{n^2}$

$\boldsymbol{r} := \boldsymbol{c} \odot \boldsymbol{w}$ ▷ *Component-wise multiplication*
for $i = 0, 1, \ldots, \lfloor \log_2 n \rfloor - 1$ ***do***
  $\boldsymbol{r} := \boldsymbol{r} + \mathrm{rot}(\boldsymbol{r}, n \cdot 2^i)$
end for
return $\boldsymbol{r} + \boldsymbol{b}$

Algorithm 2 Linear layer with repeated input

Input: Repeated ciphertext $\boldsymbol{c} \in \mathbb{R}^{n^2}$, row-wise weight matrix $\boldsymbol{w} \in \mathbb{R}^{n^2}$, expanded bias $\boldsymbol{b} \in \mathbb{R}^{n^2}$, dimension n
Output: Expanded encoding of $W\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^{n^2}$

$\boldsymbol{r} \leftarrow \boldsymbol{c} \odot \boldsymbol{w}$
for $i = 0, 1, \ldots, \lfloor \log_2 n \rfloor - 1$ **do**
  $\boldsymbol{r} \leftarrow \boldsymbol{r} + \mathrm{rot}(\boldsymbol{r},\, 2^i)$
end for
$\boldsymbol{m} \leftarrow (1, \underbrace{0, \ldots, 0}_{n-1}, 1, \underbrace{0, \ldots, 0}_{n-1}, \ldots) \in \mathbb{R}^{n^2}$ ▷ *Mask correct slots*
$\boldsymbol{r} \leftarrow \boldsymbol{r} \odot \boldsymbol{m}$
for $i = 0, 1, \ldots, \lfloor \log_2 n \rfloor - 1$ ***do*** ▷ *Repeat along empty slots*
  $\boldsymbol{r} \leftarrow \boldsymbol{r} + \mathrm{rot}(\boldsymbol{r},\, -2^i)$
end for
return $\boldsymbol{r} + \boldsymbol{b}$

For convolutions, we used ad-hoc source codes as their evaluation is harder to automate. However, our framework is flexible enough to allow for the automatic evaluation of convolutions by simply defining a function that handles the ⟨conv-layer⟩ tuple in the grammar presented in Figure 13.

E.0.2. Automatic evaluation of Chebyshev layers.. The evaluation of Chebyshev polynomials follows the footprint firsly introduced in [16] — which in turn implements a version of the Paterson-Stockmeyer algorithm [15] adapted for Chebyshev bases — however our approach is slightly more general as in [16] they assume to evaluate a single polynomial in all the slots of a ciphertext. On the other hand, in our construction we require to evaluate a different polynomial for each slot. Luckily, we can use the strategy described in [16, Section 3.1] and replace the usage of constants coefficents c_i to packed coefficients in a plaintext as $\boldsymbol{c}_i$. The following corollary easily follows.

Corollary E.1 (generalizes [16, Theorem 1])**.** *There exists an algorithm in CKKS to evaluate d polynomials of degree n given in Chebyshev base with $\sqrt{2n} + O(\log(n))$ non-scalar multiplications and $O(n)$ scalar multiplications, with d being the number of available slots in a CKKS ciphertext.*

Refer to 3. We remark that the cost of evaluating a single polynomial in all slots or a different polynomial, one for each slot, is the same as constants are typically encoded in

plaintexts in order to be compatible with the CKKS multiplication[6]. We therefore used such algorithm to allow for the evaluation of different polynomials in different slots. Since the original OpenFHE implementation of CKKS does not provide such an algorithm, we used a custom fork that enables the usage of the `EvalChebyshevSeriesPSBatch`.

E.0.3. Building the grammar. Given all the just presented approaches, we define a grammar in Figure 13 that will be the input to our CKKS program. In practice, given a polynomial network, we export it into a JSON file following the grammar, examples are given in the `networks/paper` folder of our public repository. Notice that, in its current state, che convolutional layer is still unsupported — and ad-hoc code must be written — as we leave the implementation of arbitrary convolution under CKKS as future work.

E.0.4. Automatic parameters selection. Given a JSON network, the program adapts the choice of the underlying CKKS scheme by first computing the required multiplicative depth. In practice, Algorithm 1 requires 1 multiplications and Algorithm 2 requires 2. On the other hand, the depth required by the evaluation of Chebyshev polynomials (Corollary E.1) depends on the degree as shown in Table 4. Given the

TABLE 4: Multiplicative depth by degree using the Paterson-Stockmeyer algorithm [15], [16]

Degree	Multiplicative Depth
3–5	4
6–13	5
14–27	6
28–59	7
60–119	8
120–247	9

overall multiplicative depth, along with a specified precision parameters (Δ, q_i), the program fixes the minimum required ring size N that ensures at least 128-bits of security according to [51].

Appendix F. SIMD Paterson-Stockmeyer

In this section we simply provide a formal definition of our modified version of the Paterson–Stockmeyer algorithm that supports the evaluation of s polynomials in the s different slots of a CKKS ciphertext (Algorithm 3).

Notice that the just presented algorithm is simply an extension of the original one, taking as input sets of coefficients instead of a single set. Also notice that, since the flow of execution of the algorithm heavily depends on the degree(s) of the polynomial(s), all the different degrees of $\boldsymbol{c}_i$ must be equal in our instantiation. Similarly, the integers k, m must also be the same for all the different polynomials.

6. Minor differences might occur in practice depending on how the multiplication algorithms are implemented for constants vs. plaintexts, but we can ignore them

Algorithm 3 SIMD Paterson–Stockmeyer algorithm

Input: Set of Chebyshev coefficients $(\boldsymbol{c}_0, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_s)$ of equal degree, encrypted vector $\boldsymbol{u}$ of size s

Output: $p_i(u_i) = \sum_{j=0}^{n} (c_i)_j \, T_j(u_i)$, for each $0 \le i < s$, with s being the number of CKKS slots.

Note: the notation used within the algorithm is local and does not coincide with the notation used elsewhere

1: Find positive integers k, m such that $k \approx \sqrt{n/2}$ and $k(2^m - 1) > n$.
2: $\widetilde{p}(x) \leftarrow p(x) + T_{k(2^m-1)}(x)$
3: Compute $\mathcal{B}_i \leftarrow \big(T_1(u_i),\, T_2(u_i),\, \ldots,\, T_k(u_i)\big)$ for each $i < s$ via the recurrence

$$T_0(u_i) = 1, \quad T_1(u_i) = u_i,$$
$$T_j(u_i) = 2u_i \cdot T_{j-1}(u_i) - T_{j-2}(u_i),$$

compute $\mathcal{G}_i \leftarrow \big(T_k(u_i),\, T_{2k}(u_i),\, T_{4k}(u_i),\, \ldots\big)$ via the doubling formula $T_{2\ell}(u_i) = 2\,T_\ell(u_i)^2 - 1$.
4: Apply [16, Lemma 1] to define $q_i(x), r_i(x)$ as

$$\widetilde{p}_i(x) \leftarrow T_{k\cdot 2^{m-1}}(x) \cdot q_i(x) + r_i(x),$$

with $\deg q_i = k(2^{m-1} - 1)$ and $\deg r_i < k \cdot 2^{m-1}$.
5: $\widetilde{r}_i(x) \leftarrow r_i(x) - T_{k(2^{m-1}-1)}(x)$.
6: Apply [16, Lemma 1] again to define $d_i(x)$ and $s_i(x)$ such that $\widetilde{r}_i(x) = d_i(x) \cdot q_i(x) + s_i(x)$.
7: Evaluate $d_i(u_i)$ using the Chebyshev coefficients of $\boldsymbol{c}_i$ and the precomputed values $\mathcal{B}_i$
8: $\widetilde{s}_i(x) \leftarrow s_i(x) + T_{k(2^{m-1}-1)}(x)$.
9: Recursively evaluate $q_i(u_i)$ and $\widetilde{s}_i(u_i)$ at u_i (skip Steps 1–3; reuse $\mathcal{B}$ and $\mathcal{G}$).
10: Compute

$$\widetilde{p}_i(u_i) \leftarrow \big(T_{k\cdot 2^{m-1}}(u_i) + d_i(u_i)\big) \cdot q_i(u_i) + \widetilde{s}_i(u_i),$$

where $T_{k\cdot 2^{m-1}}(u_i)$ is the last entry of $\mathcal{G}_i$.
11: Compute $T_{k(2^m-1)}(u_i)$ from $\mathcal{G}_i$ using the iterative formula derived from the Chebyshev product identity $T_a\,T_b = \frac{1}{2}\big(T_{a+b} + T_{|a-b|}\big)$:

$$V_1 \leftarrow T_k(u_i),$$
$$V_j \leftarrow 2\,V_{j-1} \cdot T_{k\cdot 2^{j-1}}(u_i) - T_k(u_i), \quad j = 2, \ldots, m;$$

12: **return** $p_i(u_i) \leftarrow \widetilde{p}_i(u_i) - V_m$

⟨root⟩ ::= `[` ⟨layer-list⟩ `]`
⟨layer-list⟩ ::= ⟨layer⟩ ⟨layer-rest⟩ | ε
⟨layer-rest⟩ ::= `,` ⟨layer⟩ ⟨layer-rest⟩ | ε
⟨layer⟩ ::= ⟨linear-layer⟩ | ⟨chebyshev-layer⟩ | ⟨conv-layer⟩

⟨linear-layer⟩ ::= `[ "linear",` ⟨num-array⟩`,` ⟨num-array⟩`,` ⟨shape⟩ `]`

⟨chebyshev-layer⟩ ::= `[ "chebyshev",` ⟨num-array⟩`,` ⟨num-array⟩`,` ⟨num-array⟩`,` ⟨shape⟩ `]`

⟨conv-layer⟩ ::= `[ "conv",` ⟨shape⟩`,` ⟨integer⟩`,` ⟨integer⟩`,`
⟨num-array⟩`,` ⟨num-array⟩`,` ⟨shape⟩`,` ⟨shape2⟩ `]`

⟨num-array⟩ ::= `[` ⟨number-list⟩ `]` | `[]`
⟨number-list⟩ ::= ⟨number⟩ | ⟨number⟩ `,` ⟨number-list⟩
⟨shape⟩ ::= `[` ⟨number⟩`,` ⟨number⟩ `]`
⟨shape2⟩ ::= `[` ⟨number⟩`,` ⟨number⟩`,` ⟨number⟩`,` ⟨number⟩ `]`

⟨number⟩ ::= ⟨integer⟩ | ⟨float⟩
⟨integer⟩ ::= ⟨digits⟩ | `-` ⟨digits⟩
⟨float⟩ ::= ⟨digits⟩ `.` ⟨digits⟩ | `-` ⟨digits⟩ `.` ⟨digits⟩
⟨digits⟩ ::= ⟨digit⟩ | ⟨digit⟩ ⟨digits⟩
⟨digit⟩ ::= `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7` | `8` | `9`

Figure 13: Grammar used to build a JSON file executable in our CKKS circuit